SIMATS ENGINEERING

Saveetha Institute of Medical And Technical Sciences

Department of Computer Science and Engineering

Course Code & Name	DSA01 – C++ Programming (functions and object-oriented concepts)
Assessment	CO2 – Assignment 2: Functions and Classes in C++ – Predict, Implement, Execute, Evaluate, Diagnose, Redesign and Validate
Course Outcome Assessed	Design C++ programs by applying functions and object-oriented concepts: function prototyping, call by reference, return by reference, inline functions, default arguments, function overloading, specifying a class, member functions, access specifiers, arrays within a class, static data members and member functions, arrays of objects and friend functions.
Student Reg. No.	192473003
Student Name	Saran. S
GitHub Repository	DSA01-192473003-Assignment2 (link: https://github.com/Saran1609/DSA0114-C-PROGRAMMING-)

C++ ASSIGNMENT 2 – CO2 REPORT

Scenario – Six Small Systems, One Language Feature Each

This report works through six small systems that each depend on one idea from the CO2 syllabus: a hostel fee updater (parameter passing, Q1), an examination marks register (return by reference, Q2), a courier charge calculator (default arguments and overloading, Q3), a savings-account mini-statement (a class with an array inside it, Q4), a college bus ticket counter (static members and arrays of objects, Q5) and a train journey planner (friend and inline functions, Q6). For every question I followed the same routine: **predict** first, **implement**, **execute** with the compiler warnings switched on, **compare** the result with the prediction, and **evaluate** a design decision. Q7 then takes one of the six programs (the Q4 account), diagnoses where it gets slow, redesigns it and checks the redesign against the original.

Given information

- Nature of assignment: individual; duration one week; total marks 50 (Q1 = 6, Q2 = 4, Q3 = 6, Q4 = 6, Q5 = 7, Q6 = 8, Q7 = 13). Every task is at Bloom level K5 (Evaluate).
- Environment: g++ 13.3.0 on Ubuntu 24.04.5, x86-64, one CPU core. Every program was compiled with `g++ -std=c++17 -Wall -Wextra`; the timing programs were also compiled with `-O0` and `-O2`.
- Evidence used: program output, compiler warnings and errors (copied exactly), exact counts of elements moved (Q4, Q7) and timings. Timings use std::chrono::steady_clock and are the best of several runs; they are supporting evidence only, the counts come first.
- Predictions were worked out on paper from the C++ rules before the corresponding program was run. Where a prediction turned out wrong this report says so.
- Repository: one folder per task (Q1 to Q7) holding the .cpp files and the captured outputs, with a README giving the compile command. All source code is reproduced in Appendix A and the raw outputs in Appendix B.

(a) Understand the assignment

The paper does not just ask for programs that work. It asks me to say what will happen before running, to run it, and to explain any difference using C++ rules. The table fixes, for each question, what is being tested and what evidence I collect.

Question	C++ concept	What is evaluated	Evidence collected	Marks
Q1 – Hostel fee	Call by value / reference / pointer, default arguments	Why the stored fee did not change; which interface is safest	Predicted vs actual output, three implementations	6
Q2 – Marks register	Return by reference, dangling references	What can and cannot be returned by reference; vector growth	Compiler warning, run results, address check	4
Q3 – Courier charge	Overloading, default arguments	Why the call is ambiguous; a design with no ambiguity	Compiler error text, six predicted selections	6
Q4 – Account	Class, private array, scope resolution	11th transaction: shifting vs circular index	Output, exact shift count	6
Q5 – Bus tickets	Static members, arrays of objects	One counter shared by all objects	Output, two linker/compiler errors	7
Q6 – Journey planner	Friend and inline functions	Friend vs member vs getters, inline cost	Tests, timings at -O0 and -O2	8
Q7 – Redesign	Chosen from Q1–Q6	Diagnose, redesign, validate	Moves and time, original vs redesign	13

Unit of cost. For the performance questions (Q4, Q7) the basic operation is one array element copied from one slot to another (a “move”). For Q6 it is one call of the comparison function. For the other questions the evidence is functional (what is printed, what the compiler says) and cost is discussed as bytes copied per call.

Method for every question. (1) Predict. (2) Implement. (3) Execute with warnings on. (4) Compare prediction with result and explain every difference. (5) Answer the inquiry or design questions from the evidence. (6) Describe the alternative concept and compare time and space using Big-O.

Q1 – Call by Value, by Reference and by Pointer (6 Marks)

The hostel office applies merit (10%), sports (15%) and single-parent (25%) concessions. The first version of the function printed the right fee inside itself, yet the stored fee never changed. That symptom is the classic sign of call by value, and the questions below prove it.

Step 1: Where does each version keep its parameter?

Version	Signature	What the parameter is	Changes caller's fee?
By value	void f(long long fee, int p)	A brand-new local variable on the callee's stack frame, initialised with a copy of the caller's value	No – only the copy changes
By reference	void f(long long& fee, int p)	No new object: *fee* is another name for the caller's variable (in practice the compiler passes its address)	Yes
By pointer	void f(long long* fee, int p)	A new local variable (8 bytes) that holds the address of the caller's variable; the fee is changed through *fee	Yes
By const reference	long long f(const Student& s)	An alias that cannot be used to modify the object	No, by design

Only two versions change the caller's fee because only two of them ever touch the caller's object. The by-value version receives a copy, subtracts the concession from the copy, and the copy disappears when the function returns. A const reference should be used when the object is large and the function only reads it. A `Student` in my test is 1,096 bytes, so passing it by value would copy 1,096 bytes on every call, whereas a const reference passes only an address and the compiler rejects any attempt to modify the student. For small types such as int or long long a plain by-value parameter is just as cheap, so a const reference gains nothing there.

Step 2: Predict the output (written before running)

Walking through the program by hand: *change(a, b, &c)* receives *a* as a copy (so the caller's a stays 1), *b* as an alias (2 becomes 12) and *c* through a pointer (3 becomes 13). In *bigger(a, c)* the test 1 > 13 is false, so the function returns a reference to *q*, which is the caller's c, and the assignment writes 100 into c, not into a. The three calls of show() add x + y + z with the defaults y = 5 and z = 10.

Line	My prediction	Actual output	Difference
1	1 12 13	1 12 13	none
2	1 100	1 100	none
3	16 13 6 (then newline)	16 13 6 (then newline)	none

Call	Arguments supplied	Default arguments used	Sum
show(1)	x = 1	y = 5 and z = 10	16
show(1, 2)	x = 1, y = 2	z = 10	13
show(1, 2, 3)	all three	none	6

There was no difference between prediction and result. The variables that changed were b (by reference) and c (by pointer) in the first call, and only c in the second, because *bigger* returned an lvalue referring to c.

Step 3: Complete the code

TODO 1 is one line, `fee -= fee * percent / 100;`, placed in the reference version. Output of the completed program:

```
after call by value : 45000
after call by reference: 40500
```

The paper prints the two expected lines joined together; that is only a line break lost in the PDF, the real program prints each on its own line because of the '\n'. The by-value call leaves the fee at 45000 because *fee* inside applyByValue is a copy: 45000 − 4500 = 40500 is computed and then thrown away with the copy.

Step 4: Three implementations, predicted and observed

Predicted stored fee after each call, starting from 45000 and applying 10% each time: by value 45000 (unchanged), by reference 40500, by pointer 36450 (10% of 40500 is 4050). The program also checks whether the parameter is the same object as the caller's variable.

```
start: 45000
  inside byValue  : local fee = 40500, same object as caller? no
  stored after byValue: 45000
  inside byRef    : fee = 40500, same object as caller? yes
  stored after byRef  : 40500
  inside byPtr    : *fee = 36450
  stored after byPtr  : 36450

Stacking (merit 10, sports 15, single parent 25) on 45000
  merit -> sports -> single parent : 25819
  single parent -> sports -> merit : 25820

sizeof(long long) = 8 bytes, sizeof(Student) = 1096 bytes
readFee(const Student&) = 45000 (no copy of 1096 bytes)
```

Call	Predicted stored fee	Observed	Same object as caller?
byValue	45000	45000	no
byRef	40500	40500	yes
byPtr	36450	36450	(pointer holds its address)

Design decision: change the fee in place or return the new fee?

Several concessions can apply to one student, so I tried the three in both orders on 45000. Merit, sports, single parent gives **25819**; single parent, sports, merit gives **25820**. The one-rupee difference is real: it comes from integer division discarding fractions at each step (8606.25 becomes 8606 in one order, 2868.8 becomes 2868 in the other). So the order matters and must be a deliberate rule.

My choice: return the new fee. I would write `[[nodiscard]] long long afterConcession(long long fee, int percent)` and let the caller store the result. My reasons: (1) the original fee stays available, which the hostel office needs for the receipt that shows the fee before and after concession; (2) stacking becomes a loop over an ordered list of percentages, so the order rule sits in one visible place; (3) calling an in-place function twice by mistake silently charges the concession twice, while a returned value that is not stored gets a compiler warning thanks to [[nodiscard]]; (4) the function is easy to test because it has no side effect; (5) returning a long long costs nothing (it travels in a register). In-place update is the better choice only when the object is large and the function is called very often, which is not the case for a fee. The by-reference version is still the right tool to demonstrate the language feature, which is why it is the one completed above.

Alternative concept and complexity analysis

The alternative interface is `long long applyConcession(long long fee, int percent)` used as `fee = applyConcession(fee, 10);`. Its time is O(1), the same as every other version: one multiplication, one division, one subtraction. The difference is in the bytes copied per call:

Parameter type and style	Bytes copied per call	Time	Extra space
long long by value	8	O(1)	O(1)
long long by reference / pointer / const reference	8 (an address)	O(1)	O(1)
Student by value	1,096 (137 times more)	O(1), but with a large constant	O(1), 1 KB of stack per call

Parameter type and style	Bytes copied per call	Time	Extra space
Student by reference or const reference	8 (an address)	O(1)	O(1)

So for an 8-byte fee the three styles are equivalent in cost and the choice is about safety and clarity; for a 1 KB object the by-value call is about 137 times more copying for no benefit.

Q1 – Call by Value, Reference and Pointer – C++ Source Code

Compiled with g++ -std=c++17 -Wall -Wextra. Files marked EXPECTED NOT TO COMPILE/LINK are the deliberate error cases discussed in this question.

q1_predict.cpp

```
#include <iostream>

void change(int a, int& b, int* c)
{
    a += 10;
    b += 10;
    *c += 10;
}

int& bigger(int& p, int& q) { return p > q ? p : q; }

void show(int x, int y = 5, int z = 10) { std::cout << x + y + z << ' '; }

int main()
{
    int a = 1, b = 2, c = 3;
    change(a, b, &c);
    std::cout << a << ' ' << b << ' ' << c << '\n';
    bigger(a, c) = 100;
    std::cout << a << ' ' << c << '\n';
    show(1);
    show(1, 2);
    show(1, 2, 3);
    std::cout << '\n';
}
```

q1_complete.cpp

```
#include <iostream>

void applyByValue(long long fee, int percent)
{
    fee -= fee * percent / 100;   // changes only the local copy
}

void applyConcession(long long& fee, int percent)
{
    // TODO 1 (done): reduce the caller's fee by percent,
    // integer arithmetic, call by reference
    fee -= fee * percent / 100;
}

int main()
{
    long long hostelFee = 45000;
    applyByValue(hostelFee, 10);
    std::cout << "after call by value    : " << hostelFee << '\n';
    applyConcession(hostelFee, 10);
    std::cout << "after call by reference: " << hostelFee << '\n';
}
```

q1_three_ways.cpp

```
#include <iostream>
#include <vector>

struct Student { char name[64]; long long fee; int history[256]; };  // a "large" object

void byValue(long long fee, int p, const long long* callerAddr)
{
    fee -= fee * p / 100;
    std::cout << "  inside byValue : local fee = " << fee
              << ", same object as caller? "
              << (&fee == callerAddr ? "yes" : "no") << '\n';
}

void byRef(long long& fee, int p, const long long* callerAddr)
{
    fee -= fee * p / 100;
    std::cout << "  inside byRef   : fee = " << fee
              << ", same object as caller? "
```

```
              << (&fee == callerAddr ? "yes" : "no") << '\n';
}

void byPtr(long long* fee, int p)
{
    if (!fee) return;             // a pointer can be null, a reference cannot
    *fee -= *fee * p / 100;
    std::cout << "  inside byPtr   : *fee = " << *fee << '\n';
}

[[nodiscard]] long long afterConcession(long long fee, int p)
{
    return fee - fee * p / 100;
}

long long readFee(const Student& s) { return s.fee; }   // no copy, no change

int main()
{
    long long fee = 45000;
    std::cout << "start: " << fee << '\n';
    byValue(fee, 10, &fee); std::cout << "  stored after byValue: " << fee << '\n';
    byRef(fee, 10, &fee);   std::cout << "  stored after byRef  : " << fee << '\n';
    byPtr(&fee, 10);        std::cout << "  stored after byPtr  : " << fee << '\n';
    byPtr(nullptr, 10);

    std::cout << "\nStacking (merit 10, sports 15, single parent 25) on 45000\n";
    long long f1 = 45000;
    for (int p : {10, 15, 25}) f1 = afterConcession(f1, p);
    long long f2 = 45000;
    for (int p : {25, 15, 10}) f2 = afterConcession(f2, p);
    std::cout << "  merit -> sports -> single parent : " << f1 << '\n';
    std::cout << "  single parent -> sports -> merit : " << f2 << '\n';

    std::cout << "\nsizeof(long long) = " << sizeof(long long)
              << " bytes, sizeof(Student) = " << sizeof(Student) << " bytes\n";
    Student s{"Asha", 45000, {0}};
    std::cout << "readFee(const Student&) = " << readFee(s)
              << " (no copy of " << sizeof(Student) << " bytes)\n";
}
```

Q2 – Return by Reference and Its Dangers (4 Marks)

The marks register is an array of marks. `int& highest(int* m, int n)` returns a reference to the largest element so that moderation can be written as `highest(marks, 5) += 2;`. I also wrote a const reference version, a by-value version, and a deliberately wrong function that returns a reference to a local variable.

Step 1: Prediction and result

For marks 56, 91, 78, 64, 85 I predicted: after +2 moderation the 91 becomes 93; after *highest(marks,5) = 90* the 93 becomes 90 (the call is an lvalue); a copy changed by +50 does not touch the array; and after two push_back calls on a vector of capacity 3 the reference no longer points into the vector.

```
after +2 moderation : 56 93 78 64 85
after highest()=90   : 56 90 78 64 85
copy+50 = 140, array max still 90
capacity before/after push_back: 3 -> 6
reference/pointer still points into the vector? no (dangling)
index 1 still valid, v[1] = 91
```

Return type	Can the caller change the element?	Cost of the return	Observed
int&	Yes, including *highest(m,n) += 2* and *highest(m,n) = 90*	An address, no copy	93 and then 90 stored in the array
const int&	No: assignment is a compile error (I left that line commented out)	An address, no copy	read the max, array untouched
int (copy)	No: the caller gets an independent value	One int copied	copy + 50 = 140, array max still 90

All predictions matched the output. Returning a reference is what makes the call usable on the left of =: the function returns the element itself, not a value taken from it.

Step 2: Returning a reference to a local variable

`int& badRef() { int local = 77; return local; }` compiles, but g++ warns:

```
Q2/q2_dangling.cpp: In function 'int& badRef()':
Q2/q2_dangling.cpp:5:12: warning: reference to local variable 'local' returned [-Wreturn-
      local-addr]
    5 |     return local;          // reference to a local variable
      |            ^~~~~
Q2/q2_dangling.cpp:4:9: note: declared here
    4 |     int local = 77;
      |         ^~~~~
```

The variable *local* lives in badRef's stack frame, which is released on return. The reference therefore names storage that no longer belongs to the program, a **dangling reference**, and reading it is undefined behaviour. When I ran the program built from it, it ended with a segmentation fault (exit status 139) at both -O0 and -O2. I would not trust that behaviour either way: the standard promises nothing, and a different compiler or setting could just as well print 77 or rubbish. The warning is the evidence that matters.

Changed condition: the register becomes a std::vector that grows

A vector keeps its elements in one block of memory. When push_back needs more room than the current capacity, the vector allocates a larger block, moves the elements into it and frees the old block. In my test the capacity went from 3 to 6 and the saved address no longer matched the element's new address (“no (dangling)” in the output above). A reference, pointer or iterator taken before the growth still refers to the freed block. The index, on the other hand, stayed valid: v[1] was still 91.

Alternative concept and complexity analysis

If the function returns an index or a pointer, the calling code changes as follows:

```
int  i = highestIndex(marks, 5);   marks[i] += 2;     // index
int* p = highestPtr(marks, 5);     *p += 2;           // pointer
highest(marks, 5) += 2;                                // reference
```

Return	Time to find the highest	Extra space	Safe when the array becomes a growing vector?	Other remark
int&	O(n)	O(1)	No – invalid after reallocation	Shortest call syntax; cannot be null
int*	O(n)	O(1)	No – invalid after reallocation	Can signal “none” with nullptr; needs * at the call
int (index)	O(n)	O(1)	Yes – an index is still valid after growth	One extra step to read or write v[i]

The scan is the same in all three, so the time is O(n) and the space O(1). The difference is safety, and the index wins for any container that can grow. I would use the reference version only for fixed arrays or when the result is used immediately, before anything can resize the container.

Q2 – Return by Reference and Its Dangers – C++ Source Code

Compiled with g++ -std=c++17 -Wall -Wextra. Files marked EXPECTED NOT TO COMPILE/LINK are the deliberate error cases discussed in this question.

q2_register.cpp

```
#include <iostream>
#include <vector>
#include <cstdint>

int& highest(int* m, int n)                    // return by reference
{
    int idx = 0;
    for (int i = 1; i < n; ++i) if (m[i] > m[idx]) idx = i;
    return m[idx];
}

const int& highestC(const int* m, int n)       // return by const reference
{
    int idx = 0;
    for (int i = 1; i < n; ++i) if (m[i] > m[idx]) idx = i;
    return m[idx];
}

int highestCopy(const int* m, int n)           // return by value (a copy)
{
    int idx = 0;
    for (int i = 1; i < n; ++i) if (m[i] > m[idx]) idx = i;
    return m[idx];
}

int highestIndex(const int* m, int n)          // alternative: index
{
    int idx = 0;
    for (int i = 1; i < n; ++i) if (m[i] > m[idx]) idx = i;
    return idx;
}

int* highestPtr(int* m, int n) { return &m[highestIndex(m, n)]; }  // alternative: pointer

int main()
{
    int marks[5] = {56, 91, 78, 64, 85};

    highest(marks, 5) += 2;                    // moderation applied directly
    std::cout << "after +2 moderation : ";
    for (int x : marks) std::cout << x << ' ';
    std::cout << '\n';

    highest(marks, 5) = 90;                    // call on the left of =
    std::cout << "after highest()=90  : ";
    for (int x : marks) std::cout << x << ' ';
    std::cout << '\n';

    int copy = highestCopy(marks, 5);          // changing the copy does not touch the array
    copy += 50;
    int cr = highestC(marks, 5);
    std::cout << "copy+50 = " << copy << ", array max still " << cr << '\n';
    // highestC(marks, 5) = 1;                 // would NOT compile: assignment to const

    std::vector<int> v = {56, 91, 78};
    int& r = highest(v.data(), (int)v.size());
    int* p = highestPtr(v.data(), (int)v.size());
    int i = highestIndex(v.data(), (int)v.size());
    std::uintptr_t before = reinterpret_cast<std::uintptr_t>(&r);
    std::size_t capBefore = v.capacity();
    v.push_back(60); v.push_back(61);          // exceeds capacity: buffer is reallocated
    std::uintptr_t now = reinterpret_cast<std::uintptr_t>(v.data() + i);
    std::cout << "capacity before/after push_back: " << capBefore
              << " -> " << v.capacity() << '\n';
    std::cout << "reference/pointer still points into the vector? "
              << (before == now ? "yes" : "no (dangling)") << '\n';
    std::cout << "index " << i << " still valid, v[" << i << "] = " << v[i] << '\n';
    (void)p;
}
```

q2_dangling.cpp

```cpp
#include <iostream>

int& badRef()
{
    int local = 77;
    return local;               // reference to a local variable
}

int main()
{
    int& r = badRef();          // r now refers to a dead stack slot
    std::cout << "value read through the dangling reference: " << r << '\n';
}
```

Q3 – Default Arguments and Function Overloading (6 Marks)

The developer wrote *charge(double)*, *charge(double, bool)* and *charge(double, bool = false, bool = false)*. I reproduced exactly that set (rates: 60 per kg, minimum 30, express × 1.5, insurance + 25) and compiled it.

Step 1: The compiler error

```
Q3/q3_ambiguous.cpp: In function 'int main()':
Q3/q3_ambiguous.cpp:9:24: error: call of overloaded 'charge(double)' is ambiguous
    9 |     std::cout << charge(2.5) << '\n';
      |                  ~~~~~~^~~~
Q3/q3_ambiguous.cpp:3:8: note: candidate: 'double charge(double)'
    3 | double charge(double weight)                              { return weight * 60; }
      |        ^~~~~~
Q3/q3_ambiguous.cpp:5:8: note: candidate: 'double charge(double, bool, bool)'
    5 | double charge(double weight, bool express = false, bool insured = false)
      |        ^~~~~~
Q3/q3_ambiguous.cpp:10:24: error: call of overloaded 'charge(double, bool)' is ambiguous
   10 |     std::cout << charge(2.5, true) << '\n';
      |                  ~~~~~~^~~~~~~~~~
Q3/q3_ambiguous.cpp:4:8: note: candidate: 'double charge(double, bool)'
    4 | double charge(double weight, bool express)                { return weight * 60 *
      |   (express ? 1.5 : 1); }
      |        ^~~~~~
Q3/q3_ambiguous.cpp:5:8: note: candidate: 'double charge(double, bool, bool)'
    5 | double charge(double weight, bool express = false, bool insured = false)
      |        ^~~~~~
```

Why it is ambiguous. For *charge(2.5)* the first function matches exactly. The third function also matches exactly once its two defaults are filled in. Default arguments are not part of how the candidates are ranked, so both are equally good and the compiler refuses to guess. *charge(2.5, true)* fails for the same reason between the second and third function. Having the same signature reachable in two ways is the actual fault.

Step 2: Redesign

I removed the one-argument and two-argument overloads and gave the defaults to **one** function per kind of input:

- `double charge(double kg, bool express = false, bool insured = false)` – weight in kilograms
- `double charge(int grams, bool express = false, bool insured = false)` – weight in grams
- `double charge(const Parcel& p)` – a Parcel object (weight and both options inside it)

Defaults sit on the right-hand parameters only. This is a rule because arguments are matched to parameters from left to right and C++ cannot skip one: if *express* had no default while *insured* did, a call *charge(2.5)* would leave *express* without a value.

Step 3: Predicted selections, then observed

How the compiler chooses: it first looks for an exact match, then a match using promotions (for example float to double, char to int), then a match using conversions (for example double to int, long to double). The best overload for every argument wins; if two are equally good the call is ambiguous.

Call	My prediction (before running)	Observed	Reason
charge(2)	int grams	int grams, 30	int is an exact match; 2 grams is below the minimum charge
charge(2.0f)	double kg	double kg, 120	float to double is a promotion, float to int is a conversion; the promotion wins
charge(2.5)	double kg	double kg, 150	exact match with double
charge(2.5, true)	double kg	double kg, 225	double exact; express adds 50%

Call	My prediction (before running)	Observed	Reason
charge(1500, true, true)	int grams	int grams, 160	1.5 kg is 90, times 1.5 is 135, plus 25 insurance
charge(Parcel{4,false,true})	Parcel	Parcel, 265	only the Parcel overload accepts a Parcel; 240 + 25

```
call 1  charge(2)                  ->  [int grams]  30
call 2  charge(2.0f)               ->  [double kg]  120
call 3  charge(2.5)                ->  [double kg]  150
call 4  charge(2.5, true)          ->  [double kg]  225
call 5  charge(1500, true, true)   ->  [int grams]  160
call 6  charge(Parcel{4,0,1})      ->  [Parcel]     265
```

All six selections matched. Two things in this output are worth noting honestly. First, *charge(2)* means 2 *grams*, which surprises a user who meant 2 kg; the design is legal but the int overload invites that mistake. Second, the redesign still has a hole: *charge(2L)* with a long argument converts equally well to int and to double, and the compiler reports it:

```
Q3/q3_still_ambiguous.cpp: In function 'int main()':
Q3/q3_still_ambiguous.cpp:5:33: error: call of overloaded 'charge(long int)' is ambiguous
    5 | int main() { std::cout << charge(2L) << '\n'; }
      |                           ~~~~~~^~~~
Q3/q3_still_ambiguous.cpp:3:8: note: candidate: 'double charge(double, bool, bool)'
    3 | double charge(double kg, bool express = false, bool insured = false) { return kg;
      }
      |        ^~~~~~
Q3/q3_still_ambiguous.cpp:4:8: note: candidate: 'double charge(int, bool, bool)'
    4 | double charge(int grams, bool express = false, bool insured = false) { return
      grams; }
      |        ^~~~~~
```

Alternative concept and complexity analysis

An alternative is a single *charge(const Parcel&)* where Parcel holds the weight and optional fields (with default member initialisers), or a function template over the weight type. Overload resolution happens entirely at compile time, so it adds no run-time cost to any design. The space copied per call differs slightly: an int is 4 bytes, a double 8 bytes, and my Parcel is 16 bytes (sizeof reported 16), which would normally be passed by const reference (8 bytes).

Design	Adding a new option (say, fragile handling)	Ambiguity risk	Weight unit
Overloads with defaults	A new parameter must be added to every overload that needs it, and each edit can create a new ambiguous call	Real (shown by charge(2L))	Encoded in the type (int = g, double = kg), easy to misuse
Single Parcel structure	Add one field with a default value; no existing call breaks	None, one function	Must be a named field, so it is explicit
Function template	Needs extra parameters or traits as well	Different kind (deduction)	Lost: the template cannot tell grams from kilograms

For this counter I would keep the three overloads from Step 2 (short common calls, as the paper asks) but I would switch to the Parcel design as soon as a third option appears, because only that design scales without touching existing calls.

Q3 – Default Arguments and Function Overloading – C++ Source Code

Compiled with g++ -std=c++17 -Wall -Wextra. Files marked EXPECTED NOT TO COMPILE/LINK are the deliberate error cases discussed in this question.

q3_ambiguous.cpp

```
// The developer's original set: this file is EXPECTED NOT TO COMPILE.
#include <iostream>

double charge(double weight) { return weight * 60; }
double charge(double weight, bool express) { return weight * 60 * (express ? 1.5 : 1); }
double charge(double weight, bool express = false, bool insured = false)
{
    return weight * 60 * (express ? 1.5 : 1) + (insured ? 25 : 0);
}

int main()
{
    std::cout << charge(2.5) << '\n';
    std::cout << charge(2.5, true) << '\n';
}
```

q3_courier.cpp

```
#include <iostream>
#include <string>

struct Parcel { double kg; bool express; bool insured; };

const double RATE_PER_KG = 60.0, MIN_CHARGE = 30.0,
             EXPRESS_FACTOR = 1.5, INSURANCE_FEE = 25.0;

// default arguments appear only here, once, from the right
double charge(double kg, bool express = false, bool insured = false)
{
    std::cout << " [double kg] ";
    double c = kg * RATE_PER_KG;
    if (c < MIN_CHARGE) c = MIN_CHARGE;
    if (express) c *= EXPRESS_FACTOR;
    if (insured) c += INSURANCE_FEE;
    return c;
}

double charge(int grams, bool express = false, bool insured = false)
{
    std::cout << " [int grams] ";
    double c = grams * RATE_PER_KG / 1000.0;
    if (c < MIN_CHARGE) c = MIN_CHARGE;
    if (express) c *= EXPRESS_FACTOR;
    if (insured) c += INSURANCE_FEE;
    return c;
}

double charge(const Parcel& p)
{
    std::cout << " [Parcel] ";
    double c = p.kg * RATE_PER_KG;
    if (c < MIN_CHARGE) c = MIN_CHARGE;
    if (p.express) c *= EXPRESS_FACTOR;
    if (p.insured) c += INSURANCE_FEE;
    return c;
}

int main()
{
    std::cout << "call 1 charge(2) ->";            std::cout << charge(2) << '\n';
    std::cout << "call 2 charge(2.0f) ->";         std::cout << charge(2.0f) << '\n';
    std::cout << "call 3 charge(2.5) ->";          std::cout << charge(2.5) << '\n';
    std::cout << "call 4 charge(2.5, true) ->";    std::cout << charge(2.5, true) << '\n';
    std::cout << "call 5 charge(1500, true, true) ->";
    std::cout << charge(1500, true, true) << '\n';
    Parcel p{4.0, false, true};
    std::cout << "call 6 charge(Parcel{4,0,1}) ->"; std::cout << charge(p) << '\n';
}
```

q3_still_ambiguous.cpp

```
// Even the redesigned set has a hole: a long argument. EXPECTED NOT TO COMPILE.
#include <iostream>

double charge(double kg, bool express = false, bool insured = false) { return kg; }
double charge(int grams, bool express = false, bool insured = false) { return grams; }

int main() { std::cout << charge(2L) << '\n'; }
```

Q4 – Specifying a Class with an Array Inside It (6 Marks)

`SavingsAccount` keeps the account number, the balance, an array of the last 10 transactions and a count, all private. The public members are *deposit()*, *withdraw()* and *miniStatement()*. Short members are defined inside the class (getBalance, deposit, and the private helper record) and the longer ones outside with the scope resolution operator (*SavingsAccount::withdraw* and *SavingsAccount::miniStatement*). The full class is in Appendix A; its shape is:

```
class SavingsAccount {
    static const int MAX_TX = 10;
    long accNo;  long balance;
    Transaction tx[MAX_TX];   int count;        // private: array inside the class
public:
    SavingsAccount(long no, long opening);
    long getBalance() const { return balance; }       // inside
    void deposit(long amt) { ... record('D', amt); }  // inside
    bool withdraw(long amt);                          // outside, uses ::
    void miniStatement() const;                       // outside, uses ::
private:
    void record(char kind, long amt);
};
```

Prediction and result

Test: opening balance 5000 and these transactions: D1000, D500, W200, D300, W150, D2000, W700, D100, W50, D400, then a withdrawal of 999999 (must be refused and not recorded), then W250 and D75. That is 12 recorded transactions. Prediction: the statement shows transactions 3 to 12 (the first two are dropped), the final balance is 8025, and, since the 11th and 12th transactions each shift 9 elements, 18 elements are shifted in total.

```
withdraw 999999 accepted? false
Mini-statement, account 1001 (last 10 transactions)
  1  W     200  balance   6300
  2  D     300  balance   6600
  3  W     150  balance   6450
  4  D    2000  balance   8450
  5  W     700  balance   7750
  6  D     100  balance   7850
  7  W      50  balance   7800
  8  D     400  balance   8200
  9  W     250  balance   7950
 10  D      75  balance   8025
final balance 8025, elements shifted: 18
```

Everything matched: the refused withdrawal printed false and does not appear, the statement starts at W200 (transaction 3), the balance is 8025 and the shift counter is 18.

Inquiry

Why the array must be private. The class promises that at most 10 transactions are stored, that they are in time order, and that each stored balance matches the real balance. If the array were public, any code could write *tx[10]* (writing past the end of the array), reorder entries or change an amount without changing the balance. Keeping it private means only deposit() and withdraw() can add entries, and they always keep these rules true.

The 11th transaction. With a shifting array the oldest entry is removed by moving the other nine one slot to the left, then the new one is stored in the last slot. That is k = 9 element moves for every transaction after the tenth, so adding is O(k). With a circular index the new transaction overwrites the oldest slot and a start index moves forward by one (wrapping at the end): zero moves, O(1). The price of the circular version is that printing must read from the start index with wrap-around, which is slightly harder to read and to get right.

Design decision: the bank wants the last 50 transactions

In my class the capacity is the single constant `MAX_TX`, so both designs change by one line (10 becomes 50) and no member function needs editing. They differ in cost: the shifting version now moves 49 elements per

transaction instead of 9, 5.4 times more work, while the circular version still moves none. Since the change is equally small, I would defend the circular design: same edit, no growth in cost. (Moving the capacity into one named constant is the design choice that makes the change small; hard-coding 10 in the loops would have meant many edits.)

Alternative concept and complexity analysis

The alternatives are a circular buffer with a start index, or a `std::deque` with push_back and pop_front. Q7 builds and measures both against the shifting array, so the numbers are given there. The summary of the analysis is:

Design	Time to add a transaction	Space	Remark
Shifting array	O(k), k = MAX_TX − 1 moves once full	MAX_TX × 24 bytes (240 for 10, 1,200 for 50)	Simple; order is always array order
Circular buffer	O(1)	Same array plus one int (the start index)	Wrap-around arithmetic when reading
std::deque	O(1) amortised	Heap blocks plus bookkeeping, more than the array	No size limit by itself: pop_front must be called manually

(sizeof(Transaction) was reported as 24 bytes on this machine: a char, padding, and two longs.)

Q4 – Class with an Array Inside It – C++ Source Code

Compiled with g++ -std=c++17 -Wall -Wextra. Files marked EXPECTED NOT TO COMPILE/LINK are the deliberate error cases discussed in this question.

q4_account.cpp

```
#include <iostream>
#include <iomanip>

struct Transaction { char kind; long amount; long balanceAfter; };

class SavingsAccount {
    static const int MAX_TX = 10;      // change to 50 for the new requirement
    long accNo;
    long balance;
    Transaction tx[MAX_TX];            // array inside the class, private
    int count;                         // how many slots are in use
public:
    static long moves;                 // counts elements shifted (for measurement)

    SavingsAccount(long no, long opening) : accNo(no), balance(opening), count(0) {}

    long getBalance() const { return balance; }      // defined inside the class

    void deposit(long amt)                           // defined inside the class
    {
        if (amt <= 0) return;
        balance += amt;
        record('D', amt);
    }

    bool withdraw(long amt);                         // defined outside
    void miniStatement() const;                      // defined outside

private:
    void record(char kind, long amt)
    {
        if (count == MAX_TX) {         // full: drop the oldest by shifting
            for (int i = 0; i < MAX_TX - 1; ++i) tx[i] = tx[i + 1];
            moves += MAX_TX - 1;
            count = MAX_TX - 1;
        }
        tx[count++] = Transaction{kind, amt, balance};
    }
};

long SavingsAccount::moves = 0;

bool SavingsAccount::withdraw(long amt)
{
    if (amt <= 0 || amt > balance) return false;     // rejected, not recorded
    balance -= amt;
    record('W', amt);
    return true;
}

void SavingsAccount::miniStatement() const
{
    std::cout << "Mini-statement, account " << accNo
              << " (last " << count << " transactions)\n";
    for (int i = 0; i < count; ++i)
        std::cout << std::setw(3) << i + 1 << "  " << tx[i].kind << "  "
                  << std::setw(6) << tx[i].amount << "  balance "
                  << std::setw(6) << tx[i].balanceAfter << '\n';
}

int main()
{
    SavingsAccount a(1001, 5000);
    a.deposit(1000); a.deposit(500); a.withdraw(200); a.deposit(300);
    a.withdraw(150); a.deposit(2000); a.withdraw(700); a.deposit(100);
    a.withdraw(50);  a.deposit(400);
    std::cout << "withdraw 999999 accepted? " << std::boolalpha
              << a.withdraw(999999) << '\n';
    a.withdraw(250); a.deposit(75);    // 11th and 12th transactions
    a.miniStatement();
    std::cout << "final balance " << a.getBalance()
```

```
            << ", elements shifted: " << SavingsAccount::moves << '\n';
}
```

Q5 – Static Members and Arrays of Objects (7 Marks)

The only principle supplied is that data belonging to the whole class, not to any object, should exist once however many objects are created. For the bus ticket class that data is the number of seats issued, and the capacity.

Design derived from the principle

- **Static members.** *issued* (the shared seat counter) must be a static data member: each ticket needs the same counter. *CAPACITY* is also shared and constant, so it is *static const*. *count()* is a static member function, so the transport office can call `BusTicket::count()` without any ticket. *seat* and *name* stay non-static because each ticket has its own.
- **Where a static data member is defined.** Declaring it inside the class does not create it. It must be defined exactly once outside the class, in one source file: `int BusTicket::issued = 0;`. I tested leaving this out and the linker failed (below).
- **Why a static member function cannot use this or a non-static member.** It is not called on an object, so there is no *this* pointer and no particular ticket whose *seat* it could mean. The compiler's message is below.
- **Array of objects.** BusTicket has no default constructor (its constructor takes a name and is explicit), so an array must be initialised element by element: `BusTicket trip[4] = {BusTicket("Asha"), ...}`; the constructor runs once per element, in order.
- **Objects in a loop or function.** Static data outlives every object, so the counter keeps its value after the objects are destroyed (shown in the test below).

```
Q5/q5_err_static_this.cpp: In static member function 'static int BusTicket::count()':
Q5/q5_err_static_this.cpp:6:42: error: invalid use of member 'BusTicket::seat' in static
      member function
    6 |     static int count() { return issued + seat; }
      |                                          ^~~~
Q5/q5_err_static_this.cpp:4:9: note: declared here
    4 |     int seat;
      |         ^~~~
```

```
/usr/bin/ld: /tmp/ccCYC2ZU.o: in function `BusTicket::BusTicket()':
q5_err_no_definition.cpp:(.text._ZN9BusTicketC2Ev[_ZN9BusTicketC5Ev]+0xe): undefined
      reference to `BusTicket::issued'
/usr/bin/ld: q5_err_no_definition.cpp:(.text._ZN9BusTicketC2Ev[_ZN9BusTicketC5Ev]+0x17):
      undefined reference to `BusTicket::issued'
/usr/bin/ld: /tmp/ccCYC2ZU.o: in function `BusTicket::count()':
q5_err_no_definition.cpp:(.text._ZN9BusTicket5countEv[_ZN9BusTicket5countEv]+0xa):
      undefined reference to `BusTicket::issued'
collect2: error: ld returned 1 exit status
```

Predict the output: the Token program

Prediction: the static *next* starts at 100 because its definition outside the class initialises it to 100, once, before main runs. *Token counter[3]* runs the constructor three times, giving ids 101, 102 and 103; *Token extra* is the fourth call and gets 104; issued() returns 104 − 100 = 4. So the line should read **101 103 104 4**, and the constructor ran 4 times.

```
101 103 104 4
```

No difference: the output is exactly as predicted.

Complete the code and test

TODO 1: `if (issued < CAPACITY) { ++issued; seat = issued; }`. TODO 2: `return issued;`. The output matches the expected output in the paper:

```
Asha: Seat 1
Bala: Seat 2
Chitra: Seat 3
Dev: Waiting
Issued: 3
```

Next, the tickets are created inside a function that is called twice, followed by an array of six tickets after the capacity is already used up:

```
batch 1
  X: Seat 1
  Y: Seat 2
  Issued after batch 1: 2
batch 2
  P: Seat 3
  Q: Waiting
  Issued after batch 2: 3
array of 6 after capacity reached:
  a: Waiting
  b: Waiting
  c: Waiting
  d: Waiting
  e: Waiting
  f: Waiting
```

The counter behaves as a counter of *tickets ever issued*, not of tickets currently alive. The two tickets of batch 1 are destroyed when makeBatch returns, yet batch 2 still starts from 2 issued and receives seat 3 and then the waiting list. Whether a destroyed ticket should free its seat is a business rule; if it should, a destructor would have to decrement the counter, and then copies of tickets would also have to be handled.

Alternative concept and complexity analysis

The alternatives are a global counter, or a *TicketOffice* object passed to every ticket. I wrote the second:

```
bus 1
  A: Seat 1
  B: Seat 2
  C: Waiting
bus 2
  D: Seat 1
  E: Seat 2
issued: bus1=2 bus2=2
```

Design	Time	Space	Encapsulation	Two buses at once?
Static data member	O(1)	One counter	Counter is private to the class	No – one counter for the whole class
Global variable	O(1)	One counter	Anyone can change it	No
TicketOffice object	O(1)	One counter per bus	Counter private to the office; tickets need a reference to it	Yes (shown above: both buses used seats 1 and 2)

For a single college bus the static member is the simplest correct design. As soon as the college runs two buses the static design cannot work and the TicketOffice design is needed.

Q5 – Static Members and Arrays of Objects – C++ Source Code

Compiled with g++ -std=c++17 -Wall -Wextra. Files marked EXPECTED NOT TO COMPILE/LINK are the deliberate error cases discussed in this question.

q5_predict.cpp

```
#include <iostream>

class Token {
    static int next;
    int id;
public:
    Token() { id = ++next; }
    int get() const { return id; }
    static int issued() { return next - 100; }
};

int Token::next = 100;

int main()
{
    Token counter[3];
    Token extra;
    std::cout << counter[0].get() << ' ' << counter[2].get() << ' '
              << extra.get() << ' ' << Token::issued() << '\n';
}
```

q5_bus.cpp

```
#include <iostream>
#include <string>

class BusTicket {
    static int issued;                  // shared by all BusTicket objects
    static const int CAPACITY = 3;
    int seat;
    std::string name;
public:
    explicit BusTicket(const std::string& n) : seat(0), name(n)
    {
        // TODO 1 (done): give the next seat while seats remain,
        // else stay on the waiting list
        if (issued < CAPACITY) {
            ++issued;
            seat = issued;
        }
    }

    static int count()
    {
        // TODO 2 (done)
        return issued;
    }

    void print() const
    {
        std::cout << name << ": "
                  << (seat ? "Seat " + std::to_string(seat) : std::string("Waiting"))
                  << '\n';
    }
};

int BusTicket::issued = 0;              // definition of the static data member

int main()
{
    BusTicket trip[4] = {BusTicket("Asha"), BusTicket("Bala"),
                         BusTicket("Chitra"), BusTicket("Dev")};
    for (const BusTicket& t : trip) t.print();
    std::cout << "Issued: " << BusTicket::count() << '\n';
}
```

q5_twice.cpp

```
#include <iostream>
#include <string>
class BusTicket {
```

```
    static int issued;
    static const int CAPACITY = 3;
    int seat;
    std::string name;
public:
    explicit BusTicket(const std::string& n) : seat(0), name(n)
    {
        if (issued < CAPACITY) seat = ++issued;
    }
    static int count() { return issued; }
    void print() const
    {
        std::cout << "  " << name << ": "
                  << (seat ? "Seat " + std::to_string(seat) : std::string("Waiting"))
                  << '\n';
    }
};

int BusTicket::issued = 0;

void makeBatch(const std::string& a, const std::string& b)
{
    BusTicket t1(a), t2(b);            // both destroyed when the function returns
    t1.print(); t2.print();
}

int main()
{
    std::cout << "batch 1\n"; makeBatch("X", "Y");
    std::cout << "  Issued after batch 1: " << BusTicket::count() << '\n';
    std::cout << "batch 2\n"; makeBatch("P", "Q");
    std::cout << "  Issued after batch 2: " << BusTicket::count() << '\n';

    BusTicket big[6] = {BusTicket("a"), BusTicket("b"), BusTicket("c"),
                        BusTicket("d"), BusTicket("e"), BusTicket("f")};
    std::cout << "array of 6 after capacity reached:\n";
    for (const BusTicket& t : big) t.print();
}
```

q5_office.cpp

```
// Alternative: the counter lives in a TicketOffice object, one office per bus.
#include <iostream>
#include <string>

class TicketOffice {
    int issued = 0;
    int capacity;
public:
    explicit TicketOffice(int cap) : capacity(cap) {}
    int nextSeat() { return issued < capacity ? ++issued : 0; }
    int count() const { return issued; }
};

class Ticket {
    int seat;
    std::string name;
public:
    Ticket(const std::string& n, TicketOffice& o) : seat(o.nextSeat()), name(n) {}
    void print() const
    {
        std::cout << "  " << name << ": "
                  << (seat ? "Seat " + std::to_string(seat) : std::string("Waiting"))
                  << '\n';
    }
};

int main()
{
    TicketOffice bus1(2), bus2(2);
    std::cout << "bus 1\n";
    Ticket("A", bus1).print();
    Ticket("B", bus1).print();
    Ticket("C", bus1).print();
    std::cout << "bus 2\n";
    Ticket("D", bus2).print();
    Ticket("E", bus2).print();
    std::cout << "issued: bus1=" << bus1.count() << " bus2=" << bus2.count() << '\n';
```

```
}
```

q5_err_static_this.cpp

```
// EXPECTED NOT TO COMPILE: a static member function using a non-static member
class BusTicket {
    static int issued;
    int seat;
public:
    static int count() { return issued + seat; }
};

int BusTicket::issued = 0;

int main() { return BusTicket::count(); }
```

q5_err_no_definition.cpp

```
// EXPECTED NOT TO LINK: static data member declared but never defined
#include <iostream>

class BusTicket {
    static int issued;
public:
    BusTicket() { ++issued; }
    static int count() { return issued; }
};

int main()
{
    BusTicket t;
    std::cout << BusTicket::count() << '\n';
}
```

Q6 – Friend Functions and Inline Functions (8 Marks)

`Time` keeps hours and minutes private. The planner needs the minutes between two times and which of two times is earlier, and calls these millions of times.

Design questions

- **Why a friend function?** Both times take part equally. A member function would have to be called on one of them (*a.minutesTo(b)*), which makes one object look special. A friend is an ordinary non-member function called *minutesBetween(a, b)*, symmetric in its arguments, yet allowed to read the private *h* and *m* of both.
- **Friend versus member.** A member is called through an object and has a *this* pointer; a friend has no *this*, is called by name with all operands as arguments, and is not inside the class's scope. Both have the same access to private data. A friend is not a member and does not make the data public to anyone else.
- **What inline asks for.** The keyword *inline* is a request, not an order: it invites the compiler to replace the call by the function body, saving the call overhead. The compiler may ignore it or inline functions that are not marked. (It also lets a function be defined in a header without breaking the one-definition rule.)
- **If the members were public instead.** Anyone could set minutes to 75 or hours to −3, so Time could no longer guarantee a valid time. The class could also never change its internal form (for example storing a single count of minutes since midnight) without breaking every piece of code that touched h and m.

Complete the code, and correctness check

TODO 1: `int d = (a.h*60 + a.m) - (b.h*60 + b.m); return d < 0 ? -d : d;`. TODO 2: `return a.h*60 + a.m < b.h*60 + b.m;`. By hand: 9:45 is 585 minutes, 13:10 is 790, so the difference is 205, and 13:10 is not earlier than 9:45. Output:

```
205 false
```

Additional tests, with the hand-calculated answer first and the program's result in the table:

```
same time twice  09:45/09:45: minutesBetween = 0, earlier(a,b) = false, earlier(b,a) = false
across noon      11:50/12:20: minutesBetween = 30, earlier(a,b) = true, earlier(b,a) = false
across noon rev  12:20/11:50: minutesBetween = 30, earlier(a,b) = false, earlier(b,a) = true
midnight edges   00:00/23:59: minutesBetween = 1439, earlier(a,b) = true, earlier(b,a) = false
same hour        14:05/14:50: minutesBetween = 45, earlier(a,b) = true, earlier(b,a) = false
```

Case	Hand calculation	Program	Agree?
09:45 and 09:45	difference 0; neither is earlier	0, false, false	yes
11:50 and 12:20 (across noon)	710 and 740 minutes: difference 30; first is earlier	30, true, false	yes
12:20 and 11:50	same difference 30; the first is not earlier	30, false, true	yes
00:00 and 23:59	0 and 1439: difference 1439	1439, true	yes
14:05 and 14:50	845 and 890: difference 45	45, true	yes

Growth investigation: 10,000,000 calls, with and without inline, at -O0 and -O2

I compared four ways of writing the same comparison: a friend forced not to be inlined (`__attribute__((noinline))`), an *inline* friend, a member function defined in the class, and an ordinary function using public getters. Each ran 10,000,000 calls over 1,024 pseudo-random times; the table gives the best of five runs. My expectation before running: at -O0 the keyword makes no difference (the optimiser is off); at -O2 inline should be faster than noinline; and the getter version should be slower at -O0 because it makes four extra calls per comparison. All four checksums were identical, so they computed the same thing.

Version	-O0 (ms)	-O2 (ms)
Friend, noinline	78.9	16.2

Version	-O0 (ms)	-O2 (ms)
Friend, inline	78.4	14.3
Member function	83.8	13.4
Public getters	176.6	13.7

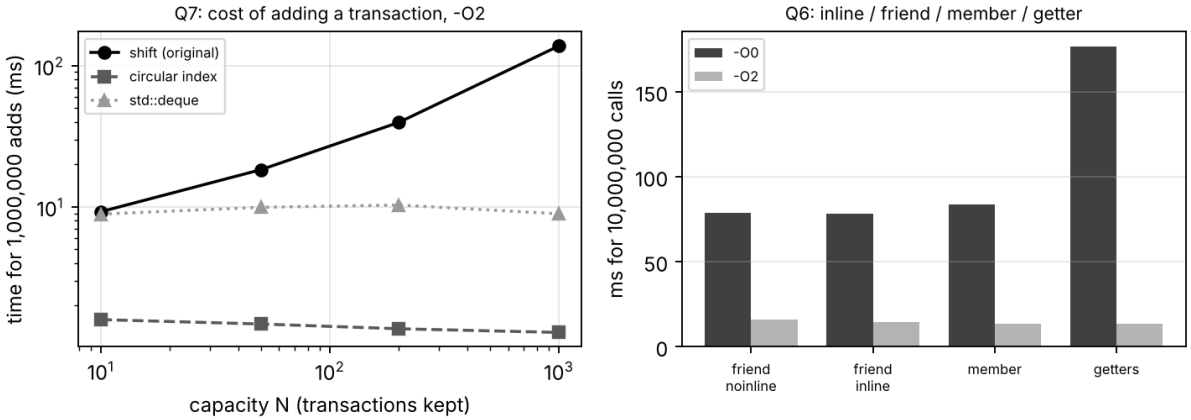


Figure 1: Left – Q7 cost of adding a transaction (explained in Q7). Right – Q6 times for 10,000,000 calls.

Explanation. At -O0 inline and noinline are the same (78.9 and 78.4 ms): without optimisation GCC does not inline, so the keyword has no effect. The getter version costs 2.2 times more (176.6 ms) because each getter is a real call. At -O2 all the versions fall to 13–16 ms (about five times faster than -O0 for the first three, thirteen times for the getters), because the optimiser inlines the small functions and keeps values in registers. Forced non-inlining is the slowest at -O2, but by only about 12% (16.2 against 14.3 ms); the call overhead is small here because the loop is dominated by memory reads. The member function, the inline friend and the getters are within 7% of one another, which is within what I would call noise for single-core timings on this machine. The honest conclusion is that **after optimisation the choice between friend, member and getters costs nothing measurable**; only the forced no-inline case lags.

Deployment decision

Keep the friend functions. The speed evidence does not separate the designs at -O2, so the decision is made on encapsulation. Friends keep the interface of Time to a constructor plus two named operations; getters would let any code read h and m and then build its own comparison, spreading knowledge of the internal layout across the program. A member version (*a.minutesTo(b)*) is equally fast and would also be acceptable, but it makes one operand special, and for *earlier* I prefer the symmetric form. If later the planner needs the hour for display, adding one getter at that time is easy. I would put the small functions in the header as *inline* so that other files can inline them at -O2.

Alternative concept and complexity analysis

Design	Time per call	Measured at -O0 / -O2	Encapsulation
Friend functions	O(1)	78 ms / 14–16 ms per 10M	Best: no extra public members; friendship is explicit and limited to named functions
Member functions (a.minutesTo(b))	O(1)	84 ms / 13 ms	Good: one operand is privileged by syntax
Public getters + ordinary function	O(1)	177 ms / 14 ms	Weakest: the layout of Time is visible to everybody

The time per call is the same O(1) after inlining (a few arithmetic operations), and no design uses extra space beyond the two ints (8 bytes) per Time object.

Q6 – Friend Functions and Inline Functions – C++ Source Code

Compiled with g++ -std=c++17 -Wall -Wextra. Files marked EXPECTED NOT TO COMPILE/LINK are the deliberate error cases discussed in this question.

q6_time.cpp

```
#include <iostream>

class Time {
    int h, m;
public:
    Time(int hh, int mm) : h(hh), m(mm) {}
    friend int minutesBetween(const Time& a, const Time& b);
    friend bool earlier(const Time& a, const Time& b);
};

int minutesBetween(const Time& a, const Time& b)
{
    // TODO 1 (done)
    int d = (a.h * 60 + a.m) - (b.h * 60 + b.m);
    return d < 0 ? -d : d;
}

bool earlier(const Time& a, const Time& b)
{
    // TODO 2 (done)
    return a.h * 60 + a.m < b.h * 60 + b.m;
}

int main()
{
    Time departure(9, 45), arrival(13, 10);
    std::cout << minutesBetween(departure, arrival) << ' '
              << std::boolalpha << earlier(arrival, departure) << '\n';
}
```

q6_tests.cpp

```
#include <iostream>

class Time {
    int h, m;
public:
    Time(int hh, int mm) : h(hh), m(mm) {}
    friend int minutesBetween(const Time&, const Time&);
    friend bool earlier(const Time&, const Time&);
};

int minutesBetween(const Time& a, const Time& b)
{
    int d = (a.h * 60 + a.m) - (b.h * 60 + b.m);
    return d < 0 ? -d : d;
}

bool earlier(const Time& a, const Time& b) { return a.h * 60 + a.m < b.h * 60 + b.m; }

void test(const char* label, Time a, Time b)
{
    std::cout << label << ": minutesBetween = " << minutesBetween(a, b)
              << ", earlier(a,b) = " << std::boolalpha << earlier(a, b)
              << ", earlier(b,a) = " << earlier(b, a) << '\n';
}

int main()
{
    test("same time twice 09:45/09:45",  Time(9, 45),  Time(9, 45));
    test("across noon 11:50/12:20",      Time(11, 50), Time(12, 20));
    test("across noon rev 12:20/11:50",  Time(12, 20), Time(11, 50));
    test("midnight edges 00:00/23:59",   Time(0, 0),   Time(23, 59));
    test("same hour 14:05/14:50",        Time(14, 5),  Time(14, 50));
}
```

q6_bench.cpp

```
#include <iostream>
#include <chrono>
#include <vector>
```

```
class Time {
    int h, m;
public:
    Time(int hh = 0, int mm = 0) : h(hh), m(mm) {}
    int hours() const { return h; }              // public getters (alternative 2)
    int mins() const { return m; }
    int minutesTo(const Time& o) const           // member function (alternative 1)
    {
        int d = (o.h * 60 + o.m) - (h * 60 + m);
        return d < 0 ? -d : d;
    }
    friend int fInline(const Time&, const Time&);
    friend int fNoInline(const Time&, const Time&);
};

inline int fInline(const Time& a, const Time& b)
{
    int d = (a.h * 60 + a.m) - (b.h * 60 + b.m);
    return d < 0 ? -d : d;
}

__attribute__((noinline)) int fNoInline(const Time& a, const Time& b)
{
    int d = (a.h * 60 + a.m) - (b.h * 60 + b.m);
    return d < 0 ? -d : d;
}

int viaGetters(const Time& a, const Time& b)     // out-of-class, uses getters only
{
    int d = (a.hours() * 60 + a.mins()) - (b.hours() * 60 + b.mins());
    return d < 0 ? -d : d;
}

template <class F>
double run(F f, const std::vector<Time>& v, long calls, long& sum)
{
    double best = 1e9;
    for (int rep = 0; rep < 5; ++rep) {
        long s = 0;
        auto t0 = std::chrono::steady_clock::now();
        for (long i = 0; i < calls; ++i) s += f(v[i & 1023], v[(i * 7 + 3) & 1023]);
        auto t1 = std::chrono::steady_clock::now();
        double ms = std::chrono::duration<double, std::milli>(t1 - t0).count();
        if (ms < best) best = ms;
        sum = s;
    }
    return best;
}

int main()
{
    std::vector<Time> v;
    unsigned x = 12345;
    for (int i = 0; i < 1024; ++i) {
        x = x * 1103515245u + 12345u;
        v.emplace_back((x >> 16) % 24, (x >> 8) % 60);
    }
    const long N = 10000000;
    long s1, s2, s3, s4;
    double a = run([](const Time& p, const Time& q) { return fNoInline(p, q); }, v, N, s1);
    double b = run([](const Time& p, const Time& q) { return fInline(p, q); },   v, N, s2);
    double c = run([](const Time& p, const Time& q) { return p.minutesTo(q); },  v, N, s3);
    double d = run([](const Time& p, const Time& q) { return viaGetters(p, q); }, v, N, s4);
    std::cout << "calls = " << N << " (checksums " << s1 << ' ' << s2 << ' '
              << s3 << ' ' << s4 << ")\n";
    std::cout << "friend, noinline : " << a << " ms\n";
    std::cout << "friend, inline   : " << b << " ms\n";
    std::cout << "member function  : " << c << " ms\n";
    std::cout << "public getters   : " << d << " ms\n";
}
```

Q7 – Diagnose, Redesign and Validate (13 Marks)

Choice of program

Question	Where it gets worse as the requirement changes	Evidence	Choose?
Q1 fee function	Nothing grows; only rounding order	25819 vs 25820	No
Q2 marks register	Reference breaks when the vector grows (a safety problem, not a speed problem)	address changed after push_back	No – fix is a one-line change to an index
Q3 courier charge	Compile-time only	ambiguity error	No
Q4 account mini-statement	Cost of each transaction grows with the capacity: 9 moves at 10, 49 at 50	18 moves for 2 extra transactions	**Yes**
Q5 bus tickets	O(1) always	–	No
Q6 time functions	Same speed at -O2	14–16 ms	No

I selected **Q4**, the shifting array, because the paper itself changes the requirement from 10 to 50 transactions and my own evidence shows the cost depends directly on that number.

A. Diagnose the limitation (2 marks)

In Q4 the program moved 18 elements for just two transactions beyond the tenth (9 each). I extended the test: for 100 consecutive transactions the shifting design moved exactly **810** elements at capacity 10 and **2,450** elements at capacity 50 (these are 90 × 9 and 50 × 49). Once the array is full, *every* new transaction moves capacity − 1 elements even though only one slot is actually new. The bottleneck is therefore the inner shifting loop in *record()*, and the characteristic of the original design that causes it is that the oldest transaction must always sit at index 0: keeping that fixed position forces the entire array to slide whenever one entry leaves.

B. Define the design objective (2 marks)

- **Correctness:** for any sequence of transactions the redesigned class must keep exactly the same last N transactions in the same order as the original (tested on 1,000 random sequences for N = 10 and for N = 50).
- **Primary objective:** element moves per added transaction = 0 once the buffer is full (the original makes N − 1).
- **Growth objective:** the time to add a transaction must not grow with N: the time at N = 1,000 must be within 2 times the time at N = 10 (the original's grows roughly in proportion to N).
- **Speed objective:** at least 10 times faster than the original at N = 50, the bank's new requirement.

C. Design the revised solution (4 marks)

What changes. The array stays inside the class and stays private, but the oldest entry no longer lives at index 0. I add one data member, *start*, the index of the oldest transaction. Adding writes to position (start + count) mod N while the buffer is filling; once full it overwrites the oldest slot (at *start*) and moves *start* forward by one, wrapping to 0 at the end. Nothing is ever moved. Reading the statement walks from *start* for *count* entries, wrapping.

```
  ShiftLog (original)                RingLog (redesign)
  +-------------------------+        +-------------------------+
  | - Tx a[N]               |        | - Tx a[N]               |
  | - int count             |        | - int start  (new)      |
  +-------------------------+        | - int count             |
  | + add(t)   O(N) moves   |        +-------------------------+
  | + snapshot()            |        | + add(t)   O(1)         |
  +-------------------------+        | + snapshot() (wraps)    |
                                     +-------------------------+

add(t):   if count < N:  p = start + count; if p >= N: p -= N; a[p] = t; count++
          else:          a[start] = t; start++; if start == N: start = 0
```

C++ features used: a class template with the capacity as a template parameter (`template <int N> class RingLog`), a private array inside the class, a private index member, and member functions defined in the class. **Extra memory:** one int (*start*), nothing else. **New trade-offs:** reading in time order needs wrap-around arithmetic (so printing and testing are a bit harder); the oldest entry is no longer at *a[0]*, so code that assumed array order cannot read the array directly; and the wrap condition is a classic place for an off-by-one error, which is why I validated it against the original. For the comparison I also wrote a *std::deque* version (*push_back*, then *pop_front* when the size exceeds N), but not as the answer: the redesign above is the hand-built one, and the deque is only a reference point.

D. Predict before implementing (1 mark)

- Moves per add after the buffer is full: 0 for the circular design, N − 1 for the original, i.e. exactly (T − N)(N − 1) moves for T adds of the original.
- The speed-up grows with N because the original's cost grows with N while the circular design's does not: I expected about 1.5–3 times at N = 10, 5–15 times at N = 50, and 50–300 times at N = 1,000 (at -O2).
- The circular design's time should be almost flat (within about 20%) from N = 10 to N = 1,000.

E. Implement and validate (2 marks)

Each version received the same 1,000,000 transactions; the table shows the best of three runs at -O2. Moves are the exact counts from the program; all three designs produced identical final contents (checksums equal).

N	Original moves	Original (ms)	Circular (ms)	Speed-up	std::deque (ms)
10	8,999,910	9.24	1.59	5.8×	8.88
50	48,997,550	18.33	1.48	12.4×	9.93
200	198,960,200	39.62	1.37	29.0×	10.30
1,000	998,001,000	137.84	1.29	106.9×	8.93

The move counts agree with (T − N)(N − 1): for N = 10, 999,990 × 9 = 8,999,910. The same test at -O0 gave speed-ups of 6.3, 32.9, 114.6 and 846 times for N = 10, 50, 200 and 1,000 (original 27.9 to 2,842.6 ms against 3.4 to 5.1 ms for the circular version). For correctness I generated 1,000 random sequences (lengths 1–300) for N = 10 and another 1,000 for N = 50 and compared the original, the circular version and the deque element by element: **1000 of 1000 and 1000 of 1000 identical**.

```
validation N=10 : 1000 of 1000 identical
validation N=50 : 1000 of 1000 identical
N=10  T=1000000  shift: 9.24025 ms (8999910 moves)  ring: 1.59008 ms (0 moves)  deque: 8.87903 ms  speed-
      up shift/ring: 5.81119  checksums equal: yes
N=50  T=1000000  shift: 18.3346 ms (48997550 moves)  ring: 1.48092 ms (0 moves)  deque: 9.92919 ms  speed-
      up shift/ring: 12.3805  checksums equal: yes
N=200  T=1000000  shift: 39.6186 ms (198960200 moves)  ring: 1.36543 ms (0 moves)  deque: 10.297 ms
      speed-up shift/ring: 29.0155  checksums equal: yes
N=1000  T=1000000  shift: 137.838 ms (998001000 moves)  ring: 1.28934 ms (0 moves)  deque: 8.93135 ms
      speed-up shift/ring: 106.906  checksums equal: yes
100 adds: shift N=10 moves = 810, shift N=50 moves = 2450
```

F. Defend the final design (2 marks)

Were the objectives met? Correctness: yes, 2,000 of 2,000 random sequences identical. Primary objective: yes, zero moves against 998,001,000 at N = 1,000. Growth objective: yes, the circular time at N = 1,000 (1.29 ms) is below the N = 10 time (1.59 ms), so it did not grow at all; the original grew from 9.2 to 137.8 ms, about 15 times. Speed objective: yes, 12.4 times at N = 50 (target 10).

Prediction versus result. N = 50 (12.4×) and N = 1,000 (107×) fell inside my predicted ranges and the circular time was flat. My prediction for N = 10 was wrong: I expected 1.5–3 times and measured 5.8 times. The ring's add is just a branch and one store, about 1.6 ns here, while the original still does a nine-element copy loop plus the counter update, so the original is relatively worse even at capacity 10 than I assumed.

New costs and where the original may still be preferable. The circular version costs one extra int, wrap-around logic when printing, and more careful testing. At the bank's current N = 10 the original costs about 9 ns per transaction (9.2 ms per million), which no customer will ever notice; its array stays in time order and is trivial to read and to debug. If the application records a few hundred transactions per day at N = 10 the original is perfectly adequate, and the simplicity argument can reasonably win. The std::deque, at about 9–10 ms per million, is level with the original at N = 10, roughly 6 times slower than the ring, and allocates from the heap, so it is not an improvement for a fixed small window.

Conclusion. The redesign meets every objective and costs one int and some wrap-around code. I would use the circular design with capacity as a template/constant parameter for the 50-transaction requirement and for any larger future window, and keep the shifting version only as the simple test oracle for the circular one, which is how it was used here.

Q7 – Diagnose, Redesign and Validate – C++ Source Code

Compiled with g++ -std=c++17 -Wall -Wextra. Files marked EXPECTED NOT TO COMPILE/LINK are the deliberate error cases discussed in this question.

q7_ring.cpp

```
#include <iostream>
#include <deque>
#include <vector>
#include <chrono>
#include <cstdlib>

struct Tx { int kind; long amount; };

template <int N>
class ShiftLog {                        // original design: shift on overflow
    Tx a[N];
    int count = 0;
public:
    static long moves;
    void add(const Tx& t)
    {
        if (count == N) {
            for (int i = 0; i < N - 1; ++i) a[i] = a[i + 1];
            moves += N - 1;
            count = N - 1;
        }
        a[count++] = t;
    }
    int snapshot(Tx* out) const
    {
        for (int i = 0; i < count; ++i) out[i] = a[i];
        return count;
    }
};
template <int N> long ShiftLog<N>::moves = 0;

template <int N>
class RingLog {                         // redesign: circular index
    Tx a[N];
    int start = 0, count = 0;
public:
    static long moves;
    void add(const Tx& t)
    {
        if (count < N) {
            int p = start + count;
            if (p >= N) p -= N;
            a[p] = t;
            ++count;
        } else {
            a[start] = t;
            if (++start == N) start = 0;
        }
    }
    int snapshot(Tx* out) const
    {
        for (int i = 0; i < count; ++i) {
            int p = start + i;
            if (p >= N) p -= N;
            out[i] = a[p];
        }
        return count;
    }
};
template <int N> long RingLog<N>::moves = 0;

template <int N>
class DequeLog {                        // library comparison
    std::deque<Tx> d;
public:
    void add(const Tx& t)
    {
        d.push_back(t);
        if ((int)d.size() > N) d.pop_front();
    }
    int snapshot(Tx* out) const
```

```
    {
        int i = 0;
        for (const Tx& t : d) out[i++] = t;
        return i;
    }
};

template <class L>
double timeIt(long T, long& check)
{
    double best = 1e9;
    for (int rep = 0; rep < 3; ++rep) {
        L log;
        auto t0 = std::chrono::steady_clock::now();
        for (long i = 0; i < T; ++i) log.add(Tx{(int)(i & 1), i});
        auto t1 = std::chrono::steady_clock::now();
        Tx out[1024];
        int n = log.snapshot(out);
        long s = 0;
        for (int i = 0; i < n; ++i) s += out[i].amount;
        check = s;
        double ms = std::chrono::duration<double, std::milli>(t1 - t0).count();
        if (ms < best) best = ms;
    }
    return best;
}

template <int N>
void bench(long T)
{
    long c1, c2, c3;
    ShiftLog<N>::moves = 0;
    double s = timeIt<ShiftLog<N>>(T, c1);
    long mv = ShiftLog<N>::moves / 3;  // 3 repetitions were run
    double r = timeIt<RingLog<N>>(T, c2);
    double q = timeIt<DequeLog<N>>(T, c3);
    std::cout << "N=" << N << " T=" << T
              << " shift: " << s << " ms (" << mv << " moves)"
              << " ring: " << r << " ms (0 moves)"
              << " deque: " << q << " ms"
              << " speed-up shift/ring: " << s / r
              << " checksums equal: " << (c1 == c2 && c2 == c3 ? "yes" : "NO") << '\n';
}

template <int N>
int validate(int trials)
{
    int ok = 0;
    for (int t = 0; t < trials; ++t) {
        ShiftLog<N> s; RingLog<N> r; DequeLog<N> d;
        int len = std::rand() % 300 + 1;
        for (int i = 0; i < len; ++i) {
            Tx x{std::rand() & 1, std::rand() % 10000};
            s.add(x); r.add(x); d.add(x);
        }
        Tx a[N], b[N], c[N];
        int na = s.snapshot(a), nb = r.snapshot(b), nc = d.snapshot(c);
        bool same = (na == nb && nb == nc);
        for (int i = 0; same && i < na; ++i)
            same = a[i].amount == b[i].amount && b[i].amount == c[i].amount
                   && a[i].kind == b[i].kind;
        ok += same;
    }
    return ok;
}

int main()
{
    std::srand(2026);
    std::cout << "validation N=10 : " << validate<10>(1000) << " of 1000 identical\n";
    std::cout << "validation N=50 : " << validate<50>(1000) << " of 1000 identical\n";
    const long T = 1000000;
    bench<10>(T); bench<50>(T); bench<200>(T); bench<1000>(T);

    // exact move counts for the report
    ShiftLog<10>::moves = 0; ShiftLog<50>::moves = 0;
    ShiftLog<10> s10; for (long i = 0; i < 100; ++i) s10.add(Tx{0, i});
    ShiftLog<50> s50; for (long i = 0; i < 100; ++i) s50.add(Tx{0, i});
```

```
    std::cout << "100 adds: shift N=10 moves = " << ShiftLog<10>::moves
              << ", shift N=50 moves = " << ShiftLog<50>::moves << '\n';
}
```

Overall Conclusion

Question	Main evidence	Decision
Q1 Fee function	By value left 45000; by reference 40500; by pointer 36450; stacking order changes the fee by 1 rupee	Return the new fee with [[nodiscard]]; keep the order of concessions as an explicit rule
Q2 Marks register	Dangling-reference warning; capacity 3 to 6 invalidated the reference	Reference for fixed arrays, index when the container can grow
Q3 Courier	Ambiguity error on charge(2.5); six selections predicted correctly; charge(2L) still ambiguous	Three overloads with defaults on one function each; move to Parcel when options grow
Q4 Account	Statement showed the last 10 of 12; 18 elements shifted	Capacity as one constant; circular index preferred
Q5 Bus tickets	101 103 104 4; seats 1–3 then waiting; counter survives destroyed objects	Static counter for one bus; TicketOffice for several
Q6 Time	205 false; at -O2 friend, member and getters all 13–16 ms	Keep friends: same speed, best encapsulation
Q7 Redesign	Moves 810 and 2,450 per 100 adds; circular 0 moves; 12.4× at N = 50, 107× at N = 1,000; 2,000/2,000 identical	Circular buffer; keep the shifting version as a test oracle

Three lessons run through the whole assignment. First, a C++ function interface is a promise about ownership and copying: value, reference and pointer all look alike in the body, but only the signature decides who can change what. Second, several features that look convenient (default arguments, return by reference) are safe only inside conditions the compiler does not check for you; overload ambiguity and dangling references showed that, and both were caught only by the warnings or errors. Third, the measurements in Q6 and Q7 agree on how to judge performance claims: the keyword inline mattered little once the optimiser was on, while a change in the algorithm (shifting versus circular index) changed the cost by two orders of magnitude.

Limitations of this study

- All timings come from one machine and one compiler (g++ 13.3.0, one core). The element-move counts are exact and machine-independent; the speed-up factors will differ elsewhere, although their trend will not.
- The compile-time behaviours (ambiguity, warnings) are those of GCC 13; another compiler may word the messages differently, and the run-time result of the dangling reference (a segmentation fault here) is not guaranteed by the language.
- The Q6 differences between inline friend, member and getters at -O2 (13.4–14.3 ms) are within the noise of single-core timing, so I draw no conclusion between them beyond “no measurable difference”.
- The Q7 workload alternates deposits and withdrawals of increasing amounts; the cost depends on the capacity, not on the amounts, so I expect the conclusion to hold for other data, but I did not test other patterns.
- I did not implement the design for waiting-list seat reuse in Q5, since the paper does not ask for it.

Mark-by-mark evidence map

Question	Marks	Evidence in this report
Q1	6	Predicted vs actual output, three implementations, stacking test, interface decision, copy-size table
Q2	4	Working functions, g++ warning, vector reallocation evidence, index/pointer comparison
Q3	6	Ambiguity error text, redesigned overloads, six predicted selections, overload ranking
Q4	6	Full class, inside/outside definitions, output, 11th-transaction analysis, capacity-50 decision

Question	Marks	Evidence in this report
Q5	7	Static design, Token prediction, completed code, function-twice test, two error messages, TicketOffice alternative
Q6	8	Friend design, completed code, correctness tests, timings at -O0 and -O2, decision
Q7	13	Diagnosis (A), objectives (B), design with diagram (C), predictions (D), validation (E), defence (F)

References

- Stroustrup, B., *The C++ Programming Language*, Addison-Wesley (functions, classes, overloading, static members, friends).
- Lippman, S., Lajoie, J., Moo, B., *C++ Primer*, Addison-Wesley (references, overload resolution, class templates).
- cppreference.com: overload resolution, default arguments, static members, *inline* specifier, vector iterator invalidation.
- Course material: CO2 Assignment 2 question paper.

Appendix A – Source Code

All programs were compiled with `g++ -std=c++17 -Wall -Wextra` (timing programs also with -O0 and -O2). Each file sits in the folder of its question in the repository. Long lines are wrapped for printing.

q1_predict.cpp – Q1

```
#include <iostream>
void change(int a, int& b, int* c)
{
    a += 10;
    b += 10; *c += 10;
}
int& bigger(int& p, int& q) { return p > q ? p : q; }
void show(int x, int y = 5, int z = 10) { std::cout << x + y + z << ' '; }
int main()
{
    int a = 1, b = 2, c = 3;
    change(a, b, &c);
    std::cout << a << ' ' << b << ' ' << c << '\n';
    bigger(a, c) = 100;
    std::cout << a << ' ' << c << '\n';
    show(1);
    show(1, 2);
    show(1, 2, 3);
    std::cout << '\n';
}
```

q1_complete.cpp – Q1

```
#include <iostream>
void applyByValue(long long fee, int percent)
{
    fee -= fee * percent / 100;   // changes only the local copy
}
void applyConcession(long long& fee, int percent)
{
    // TODO 1 (done): reduce the caller's fee by percent, integer arithmetic, call by reference
    fee -= fee * percent / 100;
}
int main()
{
    long long hostelFee = 45000;
    applyByValue(hostelFee, 10);
    std::cout << "after call by value : " << hostelFee << '\n';
    applyConcession(hostelFee, 10);
    std::cout << "after call by reference: " << hostelFee << '\n';
}
```

q1_three_ways.cpp – Q1

```
#include <iostream>
#include <vector>
struct Student { char name[64]; long long fee; int history[256]; };   // a "large" object

void byValue(long long fee, int p, const long long* callerAddr)
{
    fee -= fee * p / 100;
    std::cout << "  inside byValue  : local fee = " << fee
              << ", same object as caller? " << (&fee == callerAddr ? "yes" : "no") << '\n';
}
void byRef(long long& fee, int p, const long long* callerAddr)
{
    fee -= fee * p / 100;
    std::cout << "  inside byRef    : fee = " << fee
              << ", same object as caller? " << (&fee == callerAddr ? "yes" : "no") << '\n';
}
void byPtr(long long* fee, int p)
{
    if (!fee) return;                       // a pointer can be null, a reference cannot
    *fee -= *fee * p / 100;
    std::cout << "  inside byPtr    : *fee = " << *fee << '\n';
}
[[nodiscard]] long long afterConcession(long long fee, int p) { return fee - fee * p / 100; }
long long readFee(const Student& s) { return s.fee; }     // const reference: no copy, no change

int main()
{
    long long fee = 45000;
    std::cout << "start: " << fee << '\n';
    byValue(fee, 10, &fee);  std::cout << "  stored after byValue: " << fee << '\n';
    byRef(fee, 10, &fee);    std::cout << "  stored after byRef  : " << fee << '\n';
    byPtr(&fee, 10);         std::cout << "  stored after byPtr  : " << fee << '\n';
    byPtr(nullptr, 10);

    std::cout << "\nStacking (merit 10, sports 15, single parent 25) on 45000\n";
    long long f1 = 45000;
    for (int p : {10, 15, 25}) f1 = afterConcession(f1, p);
    long long f2 = 45000;
    for (int p : {25, 15, 10}) f2 = afterConcession(f2, p);
    std::cout << "  merit -> sports -> single parent : " << f1 << '\n';
    std::cout << "  single parent -> sports -> merit : " << f2 << '\n';

    std::cout << "\nsizeof(long long) = " << sizeof(long long)
              << " bytes, sizeof(Student) = " << sizeof(Student) << " bytes\n";
    Student s{"Asha", 45000, {0}};
    std::cout << "readFee(const Student&) = " << readFee(s) << " (no copy of "
              << sizeof(Student) << " bytes)\n";
}
```

q2_register.cpp – Q2

```
#include <iostream>
#include <vector>
#include <cstdint>

int& highest(int* m, int n)                 // return by reference
{
    int idx = 0;
    for (int i = 1; i < n; ++i) if (m[i] > m[idx]) idx = i;
    return m[idx];
}
const int& highestC(const int* m, int n)    // return by const reference
{
    int idx = 0;
    for (int i = 1; i < n; ++i) if (m[i] > m[idx]) idx = i;
    return m[idx];
}
int highestCopy(const int* m, int n)        // return by value (a copy)
{
    int idx = 0;
    for (int i = 1; i < n; ++i) if (m[i] > m[idx]) idx = i;
    return m[idx];
}
int highestIndex(const int* m, int n)       // alternative: index
{
    int idx = 0;
    for (int i = 1; i < n; ++i) if (m[i] > m[idx]) idx = i;
    return idx;
}
int* highestPtr(int* m, int n) { return &m[highestIndex(m, n)]; }   // alternative: pointer

int main()
{
    int marks[5] = {56, 91, 78, 64, 85};
    highest(marks, 5) += 2;                 // moderation applied directly
    std::cout << "after +2 moderation : ";
    for (int x : marks) std::cout << x << ' ';
    std::cout << '\n';

    highest(marks, 5) = 90;                 // call on the left of =
    std::cout << "after highest()=90   : ";
    for (int x : marks) std::cout << x << ' ';
    std::cout << '\n';

    int copy = highestCopy(marks, 5);       // changing the copy does not touch the array
    copy += 50;
    int cr = highestC(marks, 5);
    std::cout << "copy+50 = " << copy << ", array max still " << cr << '\n';
    // highestC(marks, 5) = 1;              // would NOT compile: assignment to const

    std::vector<int> v = {56, 91, 78};
    int& r = highest(v.data(), (int)v.size());
    int* p = highestPtr(v.data(), (int)v.size());
    int  i = highestIndex(v.data(), (int)v.size());
    std::uintptr_t before = reinterpret_cast<std::uintptr_t>(&r);
    std::size_t capBefore = v.capacity();
    v.push_back(60); v.push_back(61);       // exceeds capacity: the buffer is reallocated
    std::uintptr_t now = reinterpret_cast<std::uintptr_t>(v.data() + i);
    std::cout << "capacity before/after push_back: " << capBefore << " -> " << v.capacity() << '\n';
    std::cout << "reference/pointer still points into the vector? "
              << (before == now ? "yes" : "no (dangling)") << '\n';
    std::cout << "index " << i << " still valid, v[" << i << "] = " << v[i] << '\n';
    (void)p;
}
```

q2_dangling.cpp – Q2

```
#include <iostream>
int& badRef()
{
    int local = 77;
    return local;          // reference to a local variable
}
int main()
{
    int& r = badRef();     // r now refers to a dead stack slot
    std::cout << "value read through the dangling reference: " << r << '\n';
}
```

q3_ambiguous.cpp – Q3

```
// The developer's original set: this file is EXPECTED NOT TO COMPILE.
#include <iostream>
double charge(double weight)                              { return weight * 60; }
double charge(double weight, bool express)                { return weight * 60 * (express ? 1.5 : 1); }
double charge(double weight, bool express = false, bool insured = false)
{ return weight * 60 * (express ? 1.5 : 1) + (insured ? 25 : 0); }
int main()
{
    std::cout << charge(2.5) << '\n';
    std::cout << charge(2.5, true) << '\n';
}
```

q3_courier.cpp – Q3

```
#include <iostream>
#include <string>
struct Parcel { double kg; bool express; bool insured; };

const double RATE_PER_KG = 60.0, MIN_CHARGE = 30.0, EXPRESS_FACTOR = 1.5, INSURANCE_FEE = 25.0;

// default arguments appear only here, once, from the right
double charge(double kg, bool express = false, bool insured = false)
{
    std::cout << "  [double kg]  ";
    double c = kg * RATE_PER_KG;
    if (c < MIN_CHARGE) c = MIN_CHARGE;
    if (express) c *= EXPRESS_FACTOR;
    if (insured) c += INSURANCE_FEE;
    return c;
}
double charge(int grams, bool express = false, bool insured = false)
{
    std::cout << "  [int grams]  ";
    double c = grams * RATE_PER_KG / 1000.0;
    if (c < MIN_CHARGE) c = MIN_CHARGE;
    if (express) c *= EXPRESS_FACTOR;
    if (insured) c += INSURANCE_FEE;
    return c;
}
double charge(const Parcel& p)
{
    std::cout << "  [Parcel]     ";
    double c = p.kg * RATE_PER_KG;
    if (c < MIN_CHARGE) c = MIN_CHARGE;
    if (p.express) c *= EXPRESS_FACTOR;
    if (p.insured) c += INSURANCE_FEE;
    return c;
}
int main()
{
    std::cout << "call 1  charge(2)                  ->"; std::cout << charge(2) << '\n';
    std::cout << "call 2  charge(2.0f)               ->"; std::cout << charge(2.0f) << '\n';
    std::cout << "call 3  charge(2.5)                ->"; std::cout << charge(2.5) << '\n';
    std::cout << "call 4  charge(2.5, true)          ->"; std::cout << charge(2.5, true) << '\n';
    std::cout << "call 5  charge(1500, true, true)   ->"; std::cout << charge(1500, true, true) << '\n';
    Parcel p{4.0, false, true};
    std::cout << "call 6  charge(Parcel{4,0,1})      ->"; std::cout << charge(p) << '\n';
}
```

q3_still_ambiguous.cpp – Q3

```
// Even the redesigned set has a hole: a long argument. EXPECTED NOT TO COMPILE.
#include <iostream>
double charge(double kg, bool express = false, bool insured = false) { return kg; }
double charge(int grams, bool express = false, bool insured = false) { return grams; }
int main() { std::cout << charge(2L) << '\n'; }
```

q4_account.cpp – Q4

```
#include <iostream>
#include <iomanip>

struct Transaction { char kind; long amount; long balanceAfter; };

class SavingsAccount {
    static const int MAX_TX = 10;          // change to 50 for the new requirement
    long accNo;
    long balance;
    Transaction tx[MAX_TX];                // array inside the class, private
    int count;                             // how many slots are in use
public:
    static long moves;                     // counts elements shifted (for measurement)

    SavingsAccount(long no, long opening) : accNo(no), balance(opening), count(0) {}

    long getBalance() const { return balance; }          // defined inside the class

    void deposit(long amt)                                // defined inside the class
    {
        if (amt <= 0) return;
        balance += amt;
        record('D', amt);
    }
    bool withdraw(long amt);                              // defined outside
    void miniStatement() const;                           // defined outside
private:
    void record(char kind, long amt)
    {
        if (count == MAX_TX) {                            // full: drop the oldest by shifting
            for (int i = 0; i < MAX_TX - 1; ++i) tx[i] = tx[i + 1];
            moves += MAX_TX - 1;
            count = MAX_TX - 1;
        }
        tx[count++] = Transaction{kind, amt, balance};
    }
};
long SavingsAccount::moves = 0;

bool SavingsAccount::withdraw(long amt)
{
    if (amt <= 0 || amt > balance) return false;          // rejected, not recorded
    balance -= amt;
    record('W', amt);
    return true;
}
void SavingsAccount::miniStatement() const
{
    std::cout << "Mini-statement, account " << accNo << " (last " << count << " transactions)\n";
    for (int i = 0; i < count; ++i)
        std::cout << std::setw(3) << i + 1 << "  " << tx[i].kind << "  "
                  << std::setw(6) << tx[i].amount << "  balance " << std::setw(6)
                  << tx[i].balanceAfter << '\n';
}

int main()
{
    SavingsAccount a(1001, 5000);
    a.deposit(1000);  a.deposit(500);  a.withdraw(200);  a.deposit(300);
    a.withdraw(150);  a.deposit(2000); a.withdraw(700);  a.deposit(100);
    a.withdraw(50);   a.deposit(400);
    std::cout << "withdraw 999999 accepted? " << std::boolalpha << a.withdraw(999999) << '\n';
    a.withdraw(250);  a.deposit(75);                      // 11th and 12th transactions
    a.miniStatement();
    std::cout << "final balance " << a.getBalance() << ", elements shifted: "
              << SavingsAccount::moves << '\n';
}
```

q5_predict.cpp – Q5

```
#include <iostream>
class Token {
    static int next;
    int id;
public:
    Token() { id = ++next; }
    int get() const { return id; }
    static int issued() { return next - 100; }
};
int Token::next = 100;
int main(){
    Token counter[3];
    Token extra;
    std::cout << counter[0].get() << ' ' << counter[2].get() << ' ' << extra.get()
              << ' ' << Token::issued() << '\n';
}
```

q5_bus.cpp – Q5

```
#include <iostream>
#include <string>
class BusTicket {
    static int issued;                      // shared by all BusTicket objects
    static const int CAPACITY = 3;
    int seat;
    std::string name;
public:
    explicit BusTicket(const std::string& n) : seat(0), name(n)
    {
        // TODO 1 (done): give the next seat while seats remain, else stay on the waiting list
        if (issued < CAPACITY) {
            ++issued;
            seat = issued;
        }
    }
    static int count()
    {
        // TODO 2 (done)
        return issued;
    }
    void print() const
    {
        std::cout << name << ": "
                  << (seat ? "Seat " + std::to_string(seat) : std::string("Waiting"))
                  << '\n';
    }
};
int BusTicket::issued = 0;                  // definition of the static data member

int main()
{
    BusTicket trip[4] = {BusTicket("Asha"), BusTicket("Bala"),
                         BusTicket("Chitra"), BusTicket("Dev")};
    for (const BusTicket& t : trip) t.print();
    std::cout << "Issued: " << BusTicket::count() << '\n';
}
```

q5_twice.cpp – Q5

```
#include <iostream>
#include <string>
class BusTicket {
    static int issued;
    static const int CAPACITY = 3;
    int seat;
    std::string name;
public:
    explicit BusTicket(const std::string& n) : seat(0), name(n)
    { if (issued < CAPACITY) seat = ++issued; }
    static int count() { return issued; }
    void print() const
    { std::cout << "  " << name << ": " << (seat ? "Seat " + std::to_string(seat) :
      std::string("Waiting")) << '\n'; }
};
int BusTicket::issued = 0;

void makeBatch(const std::string& a, const std::string& b)
{
    BusTicket t1(a), t2(b);                // both destroyed when the function returns
    t1.print(); t2.print();
}
int main()
{
    std::cout << "batch 1\n"; makeBatch("X", "Y");
    std::cout << "  Issued after batch 1: " << BusTicket::count() << '\n';
    std::cout << "batch 2\n"; makeBatch("P", "Q");
    std::cout << "  Issued after batch 2: " << BusTicket::count() << '\n';
    BusTicket big[6] =
      {BusTicket("a"),BusTicket("b"),BusTicket("c"),BusTicket("d"),BusTicket("e"),BusTicket("f")};
    std::cout << "array of 6 after capacity reached:\n";
    for (const BusTicket& t : big) t.print();
}
```

q5_office.cpp – Q5

```
// Alternative: the counter lives in a TicketOffice object, one office per bus.
#include <iostream>
#include <string>
class TicketOffice {
    int issued = 0; int capacity;
public:
    explicit TicketOffice(int cap) : capacity(cap) {}
    int nextSeat() { return issued < capacity ? ++issued : 0; }
    int count() const { return issued; }
};
class Ticket {
    int seat; std::string name;
public:
    Ticket(const std::string& n, TicketOffice& o) : seat(o.nextSeat()), name(n) {}
    void print() const { std::cout << "  " << name << ": " << (seat ? "Seat " + std::to_string(seat) :
      std::string("Waiting")) << '\n'; }
};
int main()
{
    TicketOffice bus1(2), bus2(2);
    std::cout << "bus 1\n"; Ticket(  "A", bus1).print(); Ticket("B", bus1).print(); Ticket("C",
      bus1).print();
    std::cout << "bus 2\n"; Ticket(  "D", bus2).print(); Ticket("E", bus2).print();
    std::cout << "issued: bus1=" << bus1.count() << " bus2=" << bus2.count() << '\n';
}
```

q5_err_static_this.cpp – Q5

```
// EXPECTED NOT TO COMPILE: a static member function using a non-static member
class BusTicket {
    static int issued;
    int seat;
public:
    static int count() { return issued + seat; }
};
int BusTicket::issued = 0;
int main() { return BusTicket::count(); }
```

q5_err_no_definition.cpp – Q5

```
// EXPECTED NOT TO LINK: static data member declared but never defined
#include <iostream>
class BusTicket {
    static int issued;
public:
    BusTicket() { ++issued; }
    static int count() { return issued; }
};
int main() { BusTicket t; std::cout << BusTicket::count() << '\n'; }
```

q6_time.cpp – Q6

```
#include <iostream>
class Time {
    int h, m;
public:
    Time(int hh, int mm) : h(hh), m(mm) {}
    friend int minutesBetween(const Time& a, const Time& b);
    friend bool earlier(const Time& a, const Time& b);
};
int minutesBetween(const Time& a, const Time& b)
{
    // TODO 1 (done)
    int d = (a.h * 60 + a.m) - (b.h * 60 + b.m);
    return d < 0 ? -d : d;
}
bool earlier(const Time& a, const Time& b)
{
    // TODO 2 (done)
    return a.h * 60 + a.m < b.h * 60 + b.m;
}
int main()
{
    Time departure(9, 45), arrival(13, 10);
    std::cout << minutesBetween(departure, arrival) << ' '
              << std::boolalpha << earlier(arrival, departure) << '\n';
}
```

q6_tests.cpp – Q6

```
#include <iostream>
class Time {
    int h, m;
public:
    Time(int hh, int mm) : h(hh), m(mm) {}
    friend int minutesBetween(const Time&, const Time&);
    friend bool earlier(const Time&, const Time&);
};
int minutesBetween(const Time& a, const Time& b) { int d = (a.h*60+a.m)-(b.h*60+b.m); return d < 0 ? -d :
     d; }
bool earlier(const Time& a, const Time& b) { return a.h*60+a.m < b.h*60+b.m; }
void test(const char* label, Time a, Time b)
{
    std::cout << label << ": minutesBetween = " << minutesBetween(a, b)
              << ", earlier(a,b) = " << std::boolalpha << earlier(a, b)
              << ", earlier(b,a) = " << earlier(b, a) << '\n';
}
int main()
{
    test("same time twice  09:45/09:45", Time(9,45),  Time(9,45));
    test("across noon      11:50/12:20", Time(11,50), Time(12,20));
    test("across noon rev  12:20/11:50", Time(12,20), Time(11,50));
    test("midnight edges   00:00/23:59", Time(0,0),   Time(23,59));
    test("same hour        14:05/14:50", Time(14,5),  Time(14,50));
}
```

q6_bench.cpp – Q6

```
#include <iostream>
#include <chrono>
#include <vector>
class Time {
    int h, m;
public:
    Time(int hh = 0, int mm = 0) : h(hh), m(mm) {}
    int hours() const { return h; }                // public getters (alternative 2)
    int mins()  const { return m; }
    int minutesTo(const Time& o) const             // member function (alternative 1), inline in class
    { int d = (o.h*60+o.m) - (h*60+m); return d < 0 ? -d : d; }
    friend int fInline(const Time&, const Time&);
    friend int fNoInline(const Time&, const Time&);
};
inline int fInline(const Time& a, const Time& b)
{ int d = (a.h*60+a.m) - (b.h*60+b.m); return d < 0 ? -d : d; }
__attribute__((noinline)) int fNoInline(const Time& a, const Time& b)
{ int d = (a.h*60+a.m) - (b.h*60+b.m); return d < 0 ? -d : d; }
int viaGetters(const Time& a, const Time& b)       // out-of-class, uses getters only
{ int d = (a.hours()*60+a.mins()) - (b.hours()*60+b.mins()); return d < 0 ? -d : d; }

template <class F>
double run(F f, const std::vector<Time>& v, long calls, long& sum)
{
    double best = 1e9;
    for (int rep = 0; rep < 5; ++rep) {
        long s = 0;
        auto t0 = std::chrono::steady_clock::now();
        for (long i = 0; i < calls; ++i) s += f(v[i & 1023], v[(i * 7 + 3) & 1023]);
        auto t1 = std::chrono::steady_clock::now();
        double ms = std::chrono::duration<double, std::milli>(t1 - t0).count();
        if (ms < best) best = ms;
        sum = s;
    }
    return best;
}
int main()
{
    std::vector<Time> v;
    unsigned x = 12345;
    for (int i = 0; i < 1024; ++i) { x = x * 1103515245u + 12345u; v.emplace_back((x >> 16) % 24, (x >> 8)
       % 60); }
    const long N = 10000000;
    long s1, s2, s3, s4;
    double a = run([](const Time& p, const Time& q) { return fNoInline(p, q); }, v, N, s1);
    double b = run([](const Time& p, const Time& q) { return fInline(p, q); },   v, N, s2);
    double c = run([](const Time& p, const Time& q) { return p.minutesTo(q); },  v, N, s3);
    double d = run([](const Time& p, const Time& q) { return viaGetters(p, q); },v, N, s4);
    std::cout << "calls = " << N << "  (checksums " << s1 << ' ' << s2 << ' ' << s3 << ' ' << s4 << ")\n";
    std::cout << "friend, noinline : " << a << " ms\n";
    std::cout << "friend, inline   : " << b << " ms\n";
    std::cout << "member function  : " << c << " ms\n";
    std::cout << "public getters   : " << d << " ms\n";
}
```

q7_ring.cpp – Q7

```
#include <iostream>
#include <deque>
#include <vector>
#include <chrono>
#include <cstdlib>

struct Tx { int kind; long amount; };

template <int N>
class ShiftLog {                        // original design: shift on overflow
    Tx a[N]; int count = 0;
public:
    static long moves;
    void add(const Tx& t)
    {
        if (count == N) { for (int i = 0; i < N - 1; ++i) a[i] = a[i + 1]; moves += N - 1; count = N - 1;
      }
        a[count++] = t;
    }
    int snapshot(Tx* out) const { for (int i = 0; i < count; ++i) out[i] = a[i]; return count; }
};
template <int N> long ShiftLog<N>::moves = 0;

template <int N>
class RingLog {                         // redesign: circular index
    Tx a[N]; int start = 0, count = 0;
public:
    static long moves;
    void add(const Tx& t)
    {
        if (count < N) { int p = start + count; if (p >= N) p -= N; a[p] = t; ++count; }
        else { a[start] = t; if (++start == N) start = 0; }
    }
    int snapshot(Tx* out) const { for (int i = 0; i < count; ++i) { int p = start + i; if (p >= N) p -= N;
       out[i] =
      a[p]; } return count; }
};
template <int N> long RingLog<N>::moves = 0;

template <int N>
class DequeLog {                        // library comparison
    std::deque<Tx> d;
public:
    void add(const Tx& t) { d.push_back(t); if ((int)d.size() > N) d.pop_front(); }
    int snapshot(Tx* out) const { int i = 0; for (const Tx& t : d) out[i++] = t; return i; }
};

template <class L>
double timeIt(long T, long& check)
{
    double best = 1e9;
    for (int rep = 0; rep < 3; ++rep) {
        L log;
        auto t0 = std::chrono::steady_clock::now();
        for (long i = 0; i < T; ++i) log.add(Tx{(int)(i & 1), i});
        auto t1 = std::chrono::steady_clock::now();
        Tx out[1024]; int n = log.snapshot(out); long s = 0;
        for (int i = 0; i < n; ++i) s += out[i].amount;
        check = s;
        double ms = std::chrono::duration<double, std::milli>(t1 - t0).count();
        if (ms < best) best = ms;
    }
    return best;
}

template <int N>
void bench(long T)
{
    long c1, c2, c3;
    ShiftLog<N>::moves = 0;
    double s = timeIt<ShiftLog<N>>(T, c1);
    long mv = ShiftLog<N>::moves / 3;               // 3 repetitions were run
    double r = timeIt<RingLog<N>>(T, c2);
    double q = timeIt<DequeLog<N>>(T, c3);
    std::cout << "N=" << N << "  T=" << T
              << "  shift: " << s << " ms (" << mv << " moves)"
              << "  ring: " << r << " ms (0 moves)"
              << "  deque: " << q << " ms"
```

```
              << "  speed-up shift/ring: " << s / r
              << "  checksums equal: " << (c1 == c2 && c2 == c3 ? "yes" : "NO") << '\n';
}

template <int N>
int validate(int trials)
{
    int ok = 0;
    for (int t = 0; t < trials; ++t) {
        ShiftLog<N> s; RingLog<N> r; DequeLog<N> d;
        int len = std::rand() % 300 + 1;
        for (int i = 0; i < len; ++i) { Tx x{std::rand() & 1, std::rand() % 10000}; s.add(x); r.add(x);
      d.add(x); }
        Tx a[N], b[N], c[N];
        int na = s.snapshot(a), nb = r.snapshot(b), nc = d.snapshot(c);
        bool same = (na == nb && nb == nc);
        for (int i = 0; same && i < na; ++i) same = a[i].amount == b[i].amount && b[i].amount ==
      c[i].amount &&
      a[i].kind == b[i].kind;
        ok += same;
    }
    return ok;
}
int main()
{
    std::srand(2026);
    std::cout << "validation N=10 : " << validate<10>(1000) << " of 1000 identical\n";
    std::cout << "validation N=50 : " << validate<50>(1000) << " of 1000 identical\n";
    const long T = 1000000;
    bench<10>(T); bench<50>(T); bench<200>(T); bench<1000>(T);
    // exact move counts for the report
    ShiftLog<10>::moves = 0; ShiftLog<50>::moves = 0;
    ShiftLog<10> s10; for (long i = 0; i < 100; ++i) s10.add(Tx{0, i});
    ShiftLog<50> s50; for (long i = 0; i < 100; ++i) s50.add(Tx{0, i});
    std::cout << "100 adds: shift N=10 moves = " << ShiftLog<10>::moves
              << ", shift N=50 moves = " << ShiftLog<50>::moves << '\n';
}
```

Appendix B – Raw Program Output and Compiler Messages

Q1 q1_predict

```
1 12 13
1 100
16 13 6
```

Q7 q7_ring at -O0 (for the figures quoted in E)

```
validation N=10 : 1000 of 1000 identical
validation N=50 : 1000 of 1000 identical
N=10  T=1000000  shift: 27.9092 ms (8999910 moves)  ring: 4.40714 ms (0 moves)  deque: 24.5255 ms  speed-
      up
    shift/ring: 6.33271  checksums equal: yes
N=50  T=1000000  shift: 169.042 ms (48997550 moves)  ring: 5.13262 ms (0 moves)  deque: 27.1141 ms  speed-
      up
    shift/ring: 32.9348  checksums equal: yes
N=200  T=1000000  shift: 399.725 ms (198960200 moves)  ring: 3.48924 ms (0 moves)  deque: 26.2768 ms
      speed-up
    shift/ring: 114.559  checksums equal: yes
N=1000  T=1000000  shift: 2842.63 ms (998001000 moves)  ring: 3.35943 ms (0 moves)  deque: 25.1636 ms
      speed-up
    shift/ring: 846.162  checksums equal: yes
100 adds: shift N=10 moves = 810, shift N=50 moves = 2450
```

Q6 q6_bench at -O0

```
calls = 10000000  (checksums 4751024124 4751024124 4751024124 4751024124)
friend, noinline : 78.9226 ms
friend, inline   : 78.4452 ms
member function  : 83.8373 ms
public getters   : 176.58 ms
```

Q6 q6_bench at -O2

```
calls = 10000000  (checksums 4751024124 4751024124 4751024124 4751024124)
friend, noinline : 16.1775 ms
friend, inline   : 14.2899 ms
member function  : 13.3511 ms
public getters   : 13.7315 ms
```

Q3 q3_ambiguous (compile error, full text)

```
Q3/q3_ambiguous.cpp: In function 'int main()':
Q3/q3_ambiguous.cpp:9:24: error: call of overloaded 'charge(double)' is ambiguous
    9 |     std::cout << charge(2.5) << '\n';
      |                  ~~~~~~^~~~
Q3/q3_ambiguous.cpp:3:8: note: candidate: 'double charge(double)'
    3 | double charge(double weight)                              { return weight * 60; }
      |        ^~~~~~
Q3/q3_ambiguous.cpp:5:8: note: candidate: 'double charge(double, bool, bool)'
    5 | double charge(double weight, bool express = false, bool insured = false)
      |        ^~~~~~
Q3/q3_ambiguous.cpp:10:24: error: call of overloaded 'charge(double, bool)' is ambiguous
   10 |     std::cout << charge(2.5, true) << '\n';
      |                  ~~~~~~^~~~~~~~~~~
Q3/q3_ambiguous.cpp:4:8: note: candidate: 'double charge(double, bool)'
    4 | double charge(double weight, bool express)                { return weight * 60 * (express ? 1.5 :
      1); }
      |        ^~~~~~
Q3/q3_ambiguous.cpp:5:8: note: candidate: 'double charge(double, bool, bool)'
    5 | double charge(double weight, bool express = false, bool insured = false)
      |        ^~~~~~
```

Q3 q3_still_ambiguous (long argument, full text)

```
Q3/q3_still_ambiguous.cpp: In function 'double charge(double, bool, bool)':
Q3/q3_still_ambiguous.cpp:3:31: warning: unused parameter 'express' [-Wunused-parameter]
    3 | double charge(double kg, bool express = false, bool insured = false) { return kg; }
      |                          ~~~~~^~~~~~~~~~~~~~~
Q3/q3_still_ambiguous.cpp:3:53: warning: unused parameter 'insured' [-Wunused-parameter]
    3 | double charge(double kg, bool express = false, bool insured = false) { return kg; }
      |                                                ~~~~~^~~~~~~~~~~~~~~
Q3/q3_still_ambiguous.cpp: In function 'double charge(int, bool, bool)':
Q3/q3_still_ambiguous.cpp:4:31: warning: unused parameter 'express' [-Wunused-parameter]
    4 | double charge(int grams, bool express = false, bool insured = false) { return grams; }
      |                          ~~~~~^~~~~~~~~~~~~~~
Q3/q3_still_ambiguous.cpp:4:53: warning: unused parameter 'insured' [-Wunused-parameter]
    4 | double charge(int grams, bool express = false, bool insured = false) { return grams; }
      |                                                ~~~~~^~~~~~~~~~~~~~~
Q3/q3_still_ambiguous.cpp: In function 'int main()':
Q3/q3_still_ambiguous.cpp:5:33: error: call of overloaded 'charge(long int)' is ambiguous
    5 | int main() { std::cout << charge(2L) << '\n'; }
      |                           ~~~~~~^~~~
Q3/q3_still_ambiguous.cpp:3:8: note: candidate: 'double charge(double, bool, bool)'
    3 | double charge(double kg, bool express = false, bool insured = false) { return kg; }
      |        ^~~~~~
Q3/q3_still_ambiguous.cpp:4:8: note: candidate: 'double charge(int, bool, bool)'
    4 | double charge(int grams, bool express = false, bool insured = false) { return grams; }
      |        ^~~~~~
```